GARRITAS STORE

Manual 3 · Entender todo Garritas

Tecnologías, inicios de clases, anotaciones, responsabilidades, diagramas y recorrido de datos del código real.

Edición de estudio y laboratorio local

Código de referencia: ceaf408d39dcfa834904ed4d7bb8a7ce31bacd21

Revisión: 24/10/2026 01:11 -05

Lectura progresiva, rutas exactas y ejemplos del proyecto real.

Índice

Índice 2

1. Cómo leer el sistema 8

2. Mapa de responsabilidades y conexiones 8

3. Tecnologías: qué hacen y por qué se usan aquí 9

- Java 21 y JVM . . . 9
- Spring Boot 4.0.8 . . . 9
- Spring Cloud 2025.1.3 . . . 9
- Maven y Wrapper . . . 9
- Lombok . . . 9
- Spring MVC y REST . . . 9
- Jackson 3 . . . 9
- Bean Validation . . . 9
- JPA/Hibernate y Spring Data . . . 10
- PostgreSQL 16 . . . 10
- Flyway . . . 10
- Eureka y LoadBalancer . . . 10
- OpenFeign y Circuit Breaker . . . 10
- Kafka 4.3.1 en KRaft . . . 10
- Spring Kafka . . . 10
- Keycloak 26.8.0/OIDC/OAuth2 . . . 10
- Spring Security Resource Server . . . 10
- Angular 22 y TypeScript . . . 10
- RxJS y HttpClient . . . 11
- keycloak-js . . . 11
- Node24/npm/Angular CLI . . . 11
- Docker y Compose . . . 11
- Nginx . . . 11
- Actuator/Micrometer . . . 11
- Prometheus/Grafana . . . 11
- Loki/Promtail/SLF4J/Logback/MDC . . . 11
- JUnit/SpringBootTest/Vitest/Playwright . . . 11
- Python y kafka-python . . . 11
- Git/GitHub Actions . . . 11
- MkDocs/Material y documentación PDF . . . 12

4. Cómo leer el inicio de una clase Java 13

5. Anotaciones usadas: referencia razonada 14

6. Modelo de datos y límites de cada base 20
Venta · HTTP 8081 / PG 15441 . . . 20
Institucion · HTTP 8082 / PG 15442 . . . 20
Envio · HTTP 8083 / PG 15443 . . . 20
Citas · HTTP 8084 / PG 15444 . . . 20
Catalogo · HTTP 8085 / PG 15445 . . . 20
Usuario · HTTP 8086 / PG 15446 . . . 21
Pago · HTTP 8087 / PG 15447 . . . 21
Bienestar · HTTP 8088 / PG 15448 . . . 21
7. Transacciones, concurrencia y dinero 22
8. Compra completa: quién llama a quién 22
9. Fallos, reintentos y límites de resiliencia 24
10. Seguridad, propiedad y CORS 24
11. Citas y envíos conectados al resto 26
12. Angular: desde el arranque a los formularios 26
13. Observabilidad y correlación 26
14. Fichas de todas las clases Java 28
CurrentUser · garritas-common . . . 28
DomainErrors · garritas-common . . . 28
HttpServices · garritas-common . . . 28
Inbox · garritas-common . . . 29
MessagingConfiguration · garritas-common . . . 29
Outbox · garritas-common . . . 29
Reference · garritas-common . . . 30
SagaEvent · garritas-common . . . 30
SecurityConfiguration · garritas-common . . . 31
ServiceClient · garritas-common . . . 31
SagaContractTest · garritas-common . . . 32
GarritasConfigApplication · garritas-config . . . 32
GarritasConfigApplicationTests · garritas-config . . . 33
GarritasEurekaApplication · garritas-eureka . . . 33
GarritasEurekaApplicationTests · garritas-eureka . . . 33
GarritasGatewayApplication · garritas-gateway . . . 34
CorrelationIdFilter · garritas-gateway . . . 34
GarritasGatewayApplicationTests · garritas-gateway . . . 34

GarritasBienestarMsApplication · garritas-bienestar-ms . . . 35
BienestarController · garritas-bienestar-ms . . . 35
EspecialidadRequest · garritas-bienestar-ms . . . 36
ProfesionalRequest · garritas-bienestar-ms . . . 36
ProfesionalResponse · garritas-bienestar-ms . . . 37
Especialidad · garritas-bienestar-ms . . . 37
Profesional · garritas-bienestar-ms . . . 38
CorrelationIdFilter · garritas-bienestar-ms . . . 38
EspecialidadRepository · garritas-bienestar-ms . . . 38
ProfesionalRepository · garritas-bienestar-ms . . . 39
BienestarService · garritas-bienestar-ms . . . 39
EspecialidadesIntegrationTests · garritas-bienestar-ms . . . 40
GarritasBienestarMsApplicationTests · garritas-bienestar-ms . . . 41
GarritasCatalogoMsApplication · garritas-catalogo-ms . . . 41
CategoriaController · garritas-catalogo-ms . . . 41
ProductoController · garritas-catalogo-ms . . . 42
CategoriaRequest · garritas-catalogo-ms . . . 43
CategoriaResponse · garritas-catalogo-ms . . . 43
ProductoRequest · garritas-catalogo-ms . . . 44
ProductoResponse · garritas-catalogo-ms . . . 44
Categoria · garritas-catalogo-ms . . . 44
Producto · garritas-catalogo-ms . . . 45
GlobalExceptionHandler · garritas-catalogo-ms . . . 45
ResourceNotFoundException · garritas-catalogo-ms . . . 46
CorrelationIdFilter · garritas-catalogo-ms . . . 46
CategoriaMapper · garritas-catalogo-ms . . . 47
ProductoMapper · garritas-catalogo-ms . . . 47
InventarioEvents · garritas-catalogo-ms . . . 48
CategoriaRepository · garritas-catalogo-ms . . . 48
ProductoRepository · garritas-catalogo-ms . . . 48
CategoriaService · garritas-catalogo-ms . . . 49
ProductoService · garritas-catalogo-ms . . . 50
GarritasCatalogoMsApplicationTests · garritas-catalogo-ms . . . 50
GarritasCitasMsApplication · garritas-citas-ms . . . 51
CitaController · garritas-citas-ms . . . 51
CitaRequest · garritas-citas-ms . . . 52
CitaResponse · garritas-citas-ms . . . 52
Cita · garritas-citas-ms . . . 53
EstadoCita · garritas-citas-ms . . . 53
GlobalExceptionHandler · garritas-citas-ms . . . 53
ResourceNotFoundException · garritas-citas-ms . . . 54
CorrelationIdFilter · garritas-citas-ms . . . 54
CitaMapper · garritas-citas-ms . . . 55

CitaRepository · garritas-citas-ms . . . 55
CitaService · garritas-citas-ms . . . 55
GarritasCitasMsApplicationTests · garritas-citas-ms . . . 56
GarritasEnvioMsApplication · garritas-envio-ms . . . 56
EnvioController · garritas-envio-ms . . . 57
EnvioRequest · garritas-envio-ms . . . 58
EnvioResponse · garritas-envio-ms . . . 58
Envio · garritas-envio-ms . . . 58
EstadoEnvio · garritas-envio-ms . . . 59
GlobalExceptionHandler · garritas-envio-ms . . . 59
ResourceNotFoundException · garritas-envio-ms . . . 60
CorrelationIdFilter · garritas-envio-ms . . . 60
EnvioMapper · garritas-envio-ms . . . 60
EnvioRepository · garritas-envio-ms . . . 61
EnvioService · garritas-envio-ms . . . 61
GarritasEnvioMsApplicationTests · garritas-envio-ms . . . 62
GarritasInstitucionMsApplication · garritas-institucion-ms . . . 62
SedeController · garritas-institucion-ms . . . 63
SedeRequest · garritas-institucion-ms . . . 63
SedeResponse · garritas-institucion-ms . . . 64
Sede · garritas-institucion-ms . . . 64
GlobalExceptionHandler · garritas-institucion-ms . . . 65
ResourceNotFoundException · garritas-institucion-ms . . . 65
CorrelationIdFilter · garritas-institucion-ms . . . 65
SedeMapper · garritas-institucion-ms . . . 66
SedeRepository · garritas-institucion-ms . . . 66
SedeService · garritas-institucion-ms . . . 67
GarritasInstitucionMsApplicationTests · garritas-institucion-ms . . . 67
GarritasPagoMsApplication · garritas-pago-ms . . . 68
PagoController · garritas-pago-ms . . . 68
PagoRequest · garritas-pago-ms . . . 68
PagoResponse · garritas-pago-ms . . . 69
EstadoPago · garritas-pago-ms . . . 69
MetodoPago · garritas-pago-ms . . . 70
Pago · garritas-pago-ms . . . 70
GlobalExceptionHandler · garritas-pago-ms . . . 70
ResourceNotFoundException · garritas-pago-ms . . . 71
CorrelationIdFilter · garritas-pago-ms . . . 71
PagoMapper · garritas-pago-ms . . . 72
PagoEvents · garritas-pago-ms . . . 72
PagoRepository · garritas-pago-ms . . . 72
PagoService · garritas-pago-ms . . . 73
GarritasPagoMsApplicationTests · garritas-pago-ms . . . 73

GarritasUsuarioMsApplication · garritas-usuario-ms . . . 74
MascotaController · garritas-usuario-ms . . . 74
MiCuentaController · garritas-usuario-ms . . . 75
UsuarioController · garritas-usuario-ms . . . 76
MascotaRequest · garritas-usuario-ms . . . 76
MascotaResponse · garritas-usuario-ms . . . 77
UsuarioRequest · garritas-usuario-ms . . . 77
UsuarioResponse · garritas-usuario-ms . . . 78
Mascota · garritas-usuario-ms . . . 78
Usuario · garritas-usuario-ms . . . 78
GlobalExceptionHandler · garritas-usuario-ms . . . 79
ResourceNotFoundException · garritas-usuario-ms . . . 79
CorrelationIdFilter · garritas-usuario-ms . . . 80
MascotaMapper · garritas-usuario-ms . . . 80
UsuarioMapper · garritas-usuario-ms . . . 80
MascotaRepository · garritas-usuario-ms . . . 81
UsuarioRepository · garritas-usuario-ms . . . 81
MascotaService · garritas-usuario-ms . . . 81
UsuarioService · garritas-usuario-ms . . . 82
GarritasUsuarioMsApplicationTests · garritas-usuario-ms . . . 83
GarritasVentaMsApplication · garritas-venta-ms . . . 83
CatalogoConsulta · garritas-venta-ms . . . 84
FeignConfiguration · garritas-venta-ms . . . 84
ProductoClient · garritas-venta-ms . . . 85
VentaController · garritas-venta-ms . . . 85
DetalleVentaRequest · garritas-venta-ms . . . 86
DetalleVentaResponse · garritas-venta-ms . . . 86
VentaRequest · garritas-venta-ms . . . 86
VentaResponse · garritas-venta-ms . . . 87
DetalleVenta · garritas-venta-ms . . . 87
EstadoVenta · garritas-venta-ms . . . 87
Venta · garritas-venta-ms . . . 88
GlobalExceptionHandler · garritas-venta-ms . . . 88
ResourceNotFoundException · garritas-venta-ms . . . 89
CorrelationIdFilter · garritas-venta-ms . . . 89
DetalleVentaMapper · garritas-venta-ms . . . 89
VentaMapper · garritas-venta-ms . . . 90
VentaEvents · garritas-venta-ms . . . 90
VentaRepository · garritas-venta-ms . . . 91
VentaService · garritas-venta-ms . . . 91
GarritasVentaMsApplicationTests · garritas-venta-ms . . . 92

15. Fichas del frontend TypeScript 93

app.config.ts . . . 93
app.ts . . . 93
api.spec.ts . . . 94
api.ts . . . 94
auth.ts . . . 94
config.ts . . . 94
domain-api.ts . . . 95
interceptor.ts . . . 95
clientes.component.ts . . . 95
productos.component.ts . . . 96
especialistas.component.ts . . . 96
cart-panel.ts . . . 96
cart-store.spec.ts . . . 97
cart-store.ts . . . 97
product-catalog.ts . . . 98
citas.component.ts . . . 98
cuenta.component.spec.ts . . . 98
cuenta.component.ts . . . 99
envios.component.ts . . . 99
pagos.component.ts . . . 99
sedes.component.ts . . . 100
main.ts . . . 101

16. Configuraciones y automatizaciones: cómo encajan 102

POM Maven . . . 102
Application y Config Repo . . . 102
Docker/Compose/Nginx . . . 103
Observabilidad . . . 104
Frontend build/runtime . . . 105
Herramientas y CI . . . 105
Identidad . . . 105

17. Catálogo HTTP por controlador 106

18. Qué no debes confundir al explicar el proyecto 108

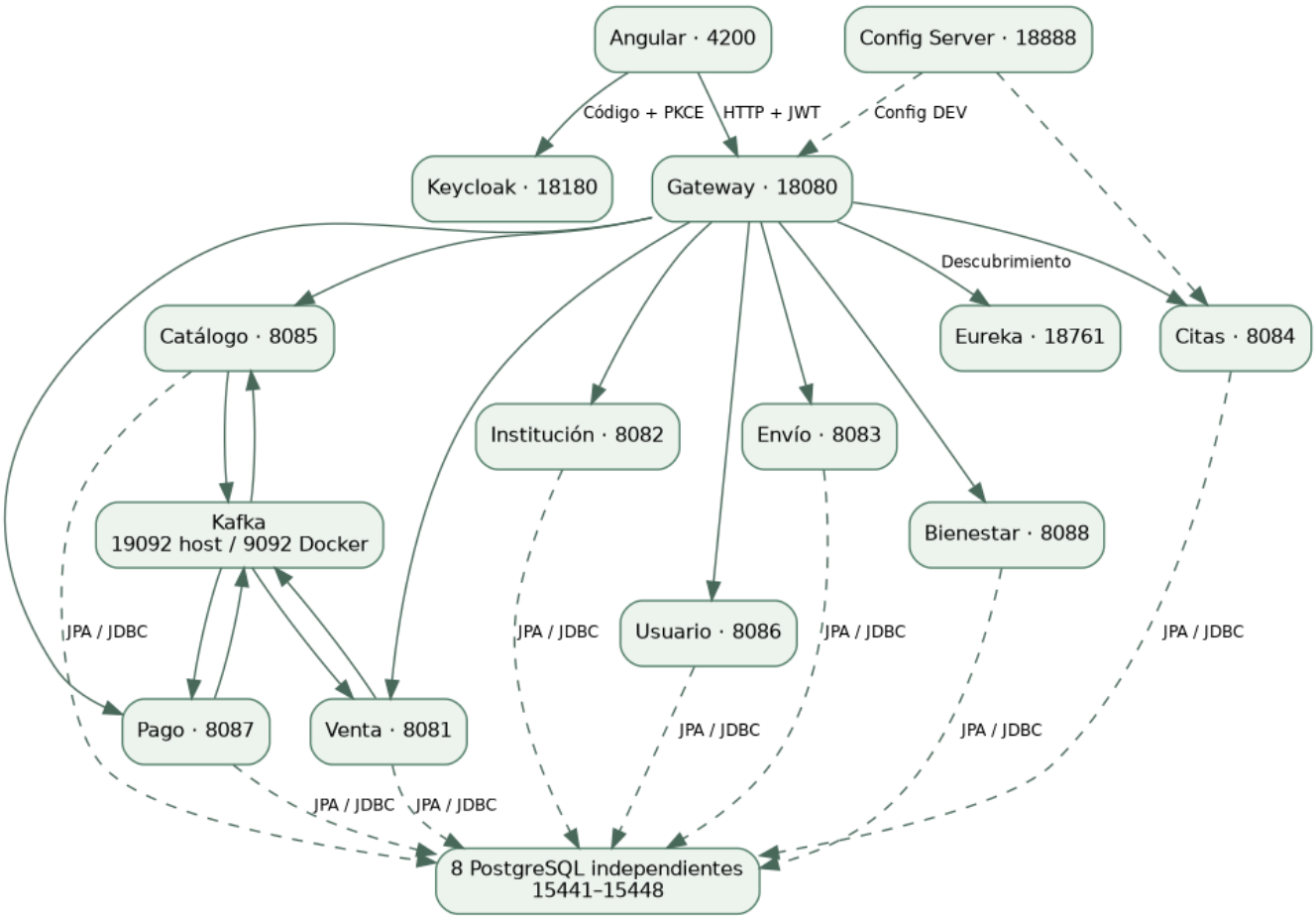
19. Lecturas y ubicación de las fuentes 108

1. Cómo leer el sistema

Este documento explica los componentes que existen en el commit ceaf408d39dcfa834904ed4d7bb8a7ce31bacd21 y la corrección de carga de perfil incluida en la edición. No es una lista de tecnologías posibles: distingue código implementado, mecanismos del framework y límites. El Manual 2 contiene el código completo por archivo; aquí se explica el porqué y la conexión, con fichas de cada fuente Java y TypeScript.

Lee primero arquitectura y flujos; después la sintaxis de clases/anotaciones y al final las fichas. Una clase no se entiende solo por su nombre: importa quién la crea, qué dependencias inyecta, qué datos recibe, qué transacción abre y quién consume su resultado.

2. Mapa de responsabilidades y conexiones



Arquitectura de ejecución. La flecha a ocho bases representa conexiones independientes, no una base compartida.

Pieza	Para qué sirve	Cómo aparece en Garritas
Angular	Interfaz ejecutada en navegador	Signals, formularios, catálogo, carrito y pantallas de los ocho dominios
Gateway MVC	Punto HTTP de entrada	Rutas Path a lb://, JWT, CORS, correlación
Config Server	Propiedades centralizadas	config-repo por nombre y perfil DEV/PROD
Eureka	Descubrimiento de instancias	Registro con nombre; LoadBalancer y Feign resuelven destino
Servicios Spring	Reglas y API por dominio	Controller → Service → Repository/Mapper
PostgreSQL	Persistencia y restricciones	Una base por dominio; no joins entre bases
Kafka	Eventos asíncronos de compra	Saga de Venta/Catálogo/Pago, tres grupos
Keycloak	Identidad y roles	OIDC, JWT, CLIENTE/ADMIN, PKCE

Pieza	Para qué sirve	Cómo aparece en Garritas
Observabilidad	Estado y diagnóstico	Actuator/Micrometer → Prometheus; logs → Loki; Grafana

Separación de dominio: Catálogo es autoridad de productos/precios/stock; Venta registra compra; Pago resuelve la simulación; Institución posee sedes; Usuario posee perfil/mascotas; Bienestar posee especialistas/especialidades; Citas posee agenda; Envío posee seguimiento. Copiar el nombre de una sede en una cita es un snapshot descriptivo, no trasladar la propiedad de la sede.

3. Tecnologías: qué hacen y por qué se usan aquí

Java 21 y JVM

Java compila fuentes .java a bytecode; la JVM ejecuta JAR. Records expresan contratos inmutables, enums limitan estados, BigDecimal evita errores binarios en dinero e Instant representa instantes UTC. JDK incluye javac; JRE solo no basta para compilar.

Spring Boot 4.0.8

Prepara contexto, servidor web embebido, auto-configuración y empaquetado ejecutable. Application no instancia manualmente cada controlador; Spring crea beans y conecta dependencias. Las propiedades importadas desde Config gobiernan puertos/datasources.

Spring Cloud 2025.1.3

BOM que coordina versiones compatibles de Config, Eureka, Gateway MVC, OpenFeign, LoadBalancer y Circuit Breaker. Un BOM administra versiones; no arranca servicios por sí mismo.

Maven y Wrapper

POM declara dependencias/plugins; Wrapper obtiene Maven de versión fijada. El reactor compila los módulos en orden. install publica un artefacto en repositorio local; package produce JAR; verify incluye pruebas y comprobaciones configuradas.

Lombok

Procesador de anotaciones que genera getters, constructores y builders durante compilación. Si el IDE muestra métodos rojos pero Maven compila, revisa soporte de anotaciones. No añade seguridad ni validación de negocio.

Spring MVC y REST

Controladores reciben HTTP y convierten JSON/DTO mediante Jackson. Spring Security filtra antes del handler. Los códigos HTTP son parte del contrato; REST no significa que cualquier entidad deba exponer todas las operaciones CRUD.

Jackson 3

tools.jackson.databind.ObjectMapper serializa SagaEvent y lee JSON de eventos. No mezcles imports com.fasterxml de otro ObjectMapper sin revisar la versión y auto-configuración. Reference ignora campos remotos que no necesita.

Bean Validation

Jakarta valida Request con Valid y restricciones. La forma de datos se controla antes del servicio; reglas como venta pagada, propiedad o especialidad activa requieren además lógica y consultas.

JPA/Hibernate y Spring Data

Entidades mapean tablas; repositorios generan persistencia. Relaciones locales usan FKs. Mapper separa API y entidad. SQL directo JDBC se usa donde los bloqueos/idempotencia/Saga necesitan control explícito.

PostgreSQL 16

Soporta transacciones ACID, restricciones, índices parciales, locks de filas y advisory lock transaccional. Se usa una base separada por dominio para impedir acoplamiento por tablas.

Flyway

Versiona DDL y datos iniciales. V se aplica una vez y guarda checksum; R se repite al cambiar. JPA ddl-auto validate comprueba el esquema, no sustituye migraciones. Los datos de demo se habilitan aparte en DEV.

Eureka y LoadBalancer

Eureka mantiene instancias; un cliente descubre y elige destino. lb:// del Gateway y el nombre Feign evitan URLs de negocio fijas. ServiceClient elige explícitamente una instancia para HTTP con JWT propagado.

OpenFeign y Circuit Breaker

ProductoClient expresa un GET declarativo. CatalogoConsulta lo envuelve en breaker; disponibilidad no se simula con precios por defecto. Timeouts acotan la espera; el breaker limita llamadas a una dependencia fallando, no repara su base.

Kafka 4.3.1 en KRaft

Broker distribuye un log particionado sin ZooKeeper en este Compose. El controlador gestiona metadatos. La clave ventaId agrupa el orden por partición; diferentes ventas pueden procesarse en paralelo. Una réplica no da tolerancia a pérdida del nodo.

Spring Kafka

KafkaTemplate publica; KafkaListener consume; group-id separa la lectura por servicio. Tres grupos leen el topic común y filtran tipos. Reintentos y DLT tratan errores técnicos; Inbox/uniqueness protegen efectos de negocio.

Keycloak 26.8.0/OIDC/OAuth2

Keycloak autentica y emite tokens. OIDC agrega identidad sobre OAuth2. Authorization Code + PKCE es adecuado para cliente público Angular. JWT firma no cifra: no se introducen secretos de negocio en sus claims.

Spring Security Resource Server

Valida token en Gateway y cada servicio. Convierte roles del realm en autoridades Spring y exige propiedad en servicio. No depende de que el usuario use la UI oficial.

Angular 22 y TypeScript

Componentes standalone y plantillas describen UI. TypeScript tipa contratos en compilación; no valida automáticamente el JSON remoto. Signals son estado reactivo, computed deriva valores y effect sincroniza

efectos como persistencia.

RxJS y HttpClient

HTTP devuelve Observables. subscribe consume respuestas/errores; App usa timer/exhaustMap para polling sin solapar solicitudes. takeUntilDestroyed libera suscripciones al destruir componente.

keycloak-js

SDK gestiona redirecciones, PKCE y renovación. Tokens en memoria; al volver a cargar se recupera sesión mediante Keycloak, no desde localStorage.

Node24/npm/Angular CLI

Node ejecuta herramientas de desarrollo, no la lógica Java. npm ci instala versiones del lockfile de forma reproducible. Build produce estáticos; ng serve es servidor de desarrollo.

Docker y Compose

Imagen empaqueta proceso y dependencias; contenedor es su ejecución; volumen conserva datos; red resuelve nombres. Compose describe el conjunto y healthcheck. DEV usa contenedores de dependencias, no los ocho JAR.

Nginx

En la alternativa frontend de producción sirve estáticos y proxifica /api/v1. El config.json relativo evita CORS adicional en ese origen. No es el servidor usado por npm start.

Actuator/Micrometer

Expone health y métricas de aplicaciones Java. Liveness pregunta por vida del proceso; readiness por disposición declarada. La configuración de Garritas no equivale a una prueba automática de todas las dependencias.

Prometheus/Grafana

Prometheus recoge series métricas; Grafana consulta y presenta paneles. Datos fuente y dashboards se aprovisionan para no configurarlos manualmente en cada instalación.

Loki/Promtail/SLF4J/Logback/MDC

Logback escribe logs, MDC añade correlación. Promtail lee archivos y envía a Loki. Loki almacena/consulta logs; Grafana Explore permite buscar. No se ha añadido tracing de spans OpenTelemetry.

JUnit/SpringBootTest/Vitest/Playwright

JUnit comprueba contexto/contrato/integración; Vitest comprueba frontend; Playwright navega con login real y prueba varios dominios/API. Cada nivel detecta fallos distintos: 200 en health no demuestra permisos ni stock.

Python y kafka-python

Las herramientas de smoke/status y el ejemplo productor/consumidor usan Python. El ejemplo usa topic aislado, no es un noveno microservicio de negocio ni reemplaza Kafka de Spring.

Git/GitHub Actions

Git conserva fuentes/commits/ramas; GitHub aloja el remoto. El workflow compila y prueba frontend. Subir código no publica automáticamente una web accesible en Internet.

MkDocs/Material y documentación PDF

MkDocs compila documentación navegable. Los PDFs de esta edición se generan con ReportLab y diagramas Graphviz; esas herramientas documentales no son dependencias de ejecución de la tienda.

4. Cómo leer el inicio de una clase Java

```
package pe.edu.upeu.garritas.institucion.service;

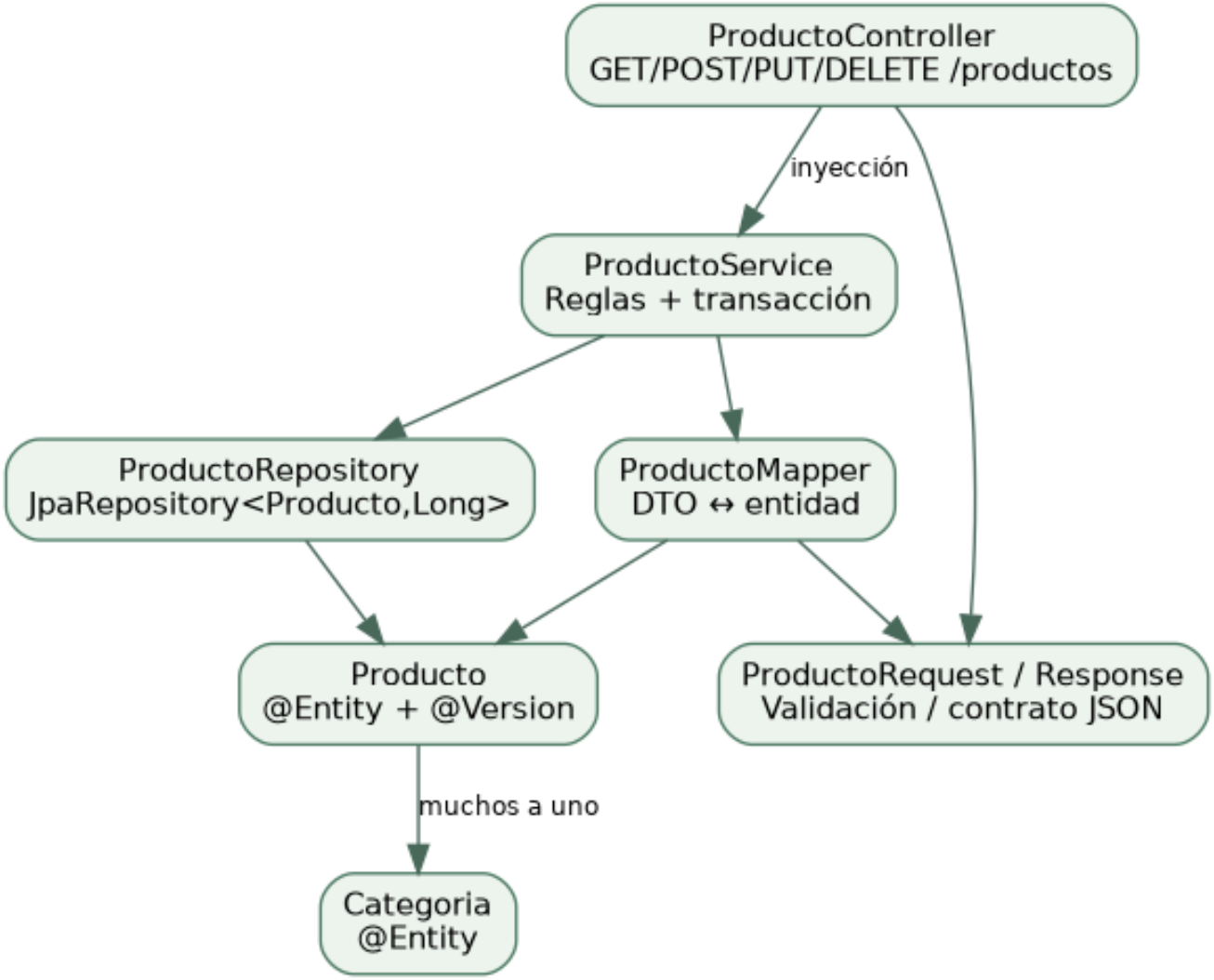
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class SedeService {
  private final SedeRepository sedeRepository;
  private final SedeMapper sedeMapper;
  // Métodos completos: ver archivo del Manual 2.
}
```

package identifica el espacio de nombres y debe coincidir con la ruta bajo src/main/java. import permite usar un tipo sin escribir su nombre completo; no descarga dependencias: eso es Maven. @Service registra lógica de aplicación. @RequiredArgsConstructor genera constructor para campos final; Spring lo usa para inyectar repositorio y mapper. public permite acceso fuera del paquete; final impide reasignar la referencia, no vuelve inmutable todo el objeto.

Una interfaz declara un contrato. ProductoRepository extends JpaRepository<Producto,Long> hereda operaciones que Spring implementa. Un record define componentes y métodos de acceso como id(), no getId(). Un enum define constantes finitas. static pertenece a la clase; private limita acceso interno. throws/throw expresan fallos, no respuestas HTTP hasta que un manejador los traduce.

DTO es un objeto de transferencia: Request describe entrada y Response salida. Una entidad contiene mapeo persistente y puede tener relaciones lazy; devolverla directamente acoplaría JSON y base. Mapper transforma, Service decide y Repository persiste. El controlador maneja HTTP y no debería realizar SQL de reserva de inventario.



Relaciones UML simplificadas y reales de Catálogo. Las flechas indican uso/inyección, no herencia en todos los casos.

5. Anotaciones usadas: referencia razonada

@AfterEach

Limpia estado después de cada caso de prueba.

@AllArgsConstructor

Genera constructor con todos los campos correspondientes.

@Autowired

Solicita inyección de un bean compatible; en pruebas permite obtener dependencias del contexto. Con un único constructor Spring puede inyectar sin escribirla.

@Bean

El valor retornado se registra y administra en el contenedor de Spring; sus dependencias llegan como parámetros/inyección.

@BeforeEach

Prepara estado antes de cada caso de prueba.

@Builder

Lombok genera un builder para construir objetos; no reemplaza invariantes del servicio.

@Column

Define nombre, nulabilidad, longitud/precisión y otras características del mapeo.

@Component

Registra un componente detectado por el escaneo, como consumidores y mappers.

@Configuration

Declara una fuente de definiciones de beans; proxyBeanMethods=false evita proxies innecesarios de métodos de configuración.

@DecimalMin

Límite inferior decimal, útil para importes. El redondeo/escala se trata aparte.

@DeleteMapping

Baja DELETE; en productos, sedes y profesionales puede ser lógica, no borrado SQL.

@Email

Valida formato básico de correo; no acredita que exista ni que pertenezca al usuario.

@EnableConfigServer

Activa endpoints de Spring Cloud Config Server que sirven propiedades por aplicación/perfil.

@EnableEurekaServer

Activa el registro de servicios Eureka en el módulo de infraestructura.

@EnableFeignClients

Escanea interfaces @FeignClient para crear proxies HTTP declarativos.

@EnableScheduling

Activa ejecución de Scheduled en el contexto.

@Entity

Entidad persistente JPA; se asocia con tabla, pero aquí Flyway administra el DDL.

@EntityGraph

Carga relaciones declaradas para una lectura concreta; evita depender de serializar relaciones lazy fuera de la transacción.

@Enumerated

Guarda enum como nombre cuando usa STRING; evita depender del ordinal.

@ExceptionHandler

Declara qué tipo de excepción maneja un método advice.

@FeignClient

Crea proxy HTTP por servicio descubierto. ProductoClient consulta Catálogo por nombre.

@Future

Exige una fecha posterior al instante de validación; Citas además valida el bloque horario.

@GeneratedValue

La base/proveedor genera el id según estrategia declarada; no fabricar ids desde Angular.

@GetMapping

Consulta GET; aquí no debe inventar cambios de negocio.

@Getter

Lombok genera getters.

@Id

Campo identificador JPA.

@Import

Registra clases de configuración externas al escaneo; así cada módulo incorpora seguridad/comunicación común explícitamente.

@JoinColumn

Declara la columna FK del lado propietario de la relación local.

@JoinTable

Declara la tabla de asociación y columnas de enlace para M:N.

@JsonIgnoreProperties

Permite ignorar propiedades JSON extra cuando se configura ignoreUnknown; Reference usa esta tolerancia.

@KafkaListener

Registra un consumidor sobre el topic declarado y el grupo del perfil. Su método debe filtrar eventos, validar y deduplicar antes de aplicar negocio.

@ManyToMany

Relaciona conjuntos mediante una tabla de unión; usado en Bienestar.

@ManyToOne

Muchas filas de la entidad apuntan a una fila relacionada dentro de la misma base.

@Max

Exige un número menor o igual al límite declarado; acota valores que el dominio acepta.

@Min

Exige un número mayor o igual al límite declarado; se usa en campos como stock, edad o cantidades según DTO.

@NoArgsConstructor

Genera constructor sin argumentos, habitual para JPA.

@NotBlank

Exige texto no nulo y con contenido distinto de espacios.

@NotEmpty

Exige colección/cadena no vacía.

@NotNull

Exige presencia de un valor; no sustituye Positive ni NotBlank.

@OneToMany

Una entidad tiene varias filas relacionadas; mappedBy marca el lado dueño inverso.

@Override

El compilador comprueba que el método implementa o sobreescribe una firma heredada.

@PathVariable

Toma un segmento de la URL y lo convierte al tipo Java declarado.

@Pattern

Exige que el texto cumpla la expresión regular declarada; por ejemplo limita valores textuales permitidos, además de no nulidad si corresponde.

@Positive

Exige número estrictamente mayor que cero; se usa en ids/cantidades.

@PostMapping

Operación POST: creación o transición explícita según ruta.

@PutMapping

Actualización PUT del recurso definido por el endpoint.

@Query

Define SQL/JPQL explícito en repositorio según nativeQuery.

@RequestBody

Deserializa el JSON recibido a DTO; la validación se activa con Valid.

@RequestHeader

Obtiene una cabecera, por ejemplo Idempotency-Key en Venta.

@RequestMapping

Declara prefijo/ruta y, si se especifica, método HTTP. Los mappings de método se combinan con el de clase.

@RequiredArgsConstructor

Lombok genera constructor para final/no nulos. Spring inyecta por ese constructor; no es un constructor escrito manualmente.

@ResponseStatus

Declara el código de respuesta del handler cuando se completa sin excepción.

@RestController

Expone métodos como handlers HTTP con serialización de respuesta; combina Controller y ResponseBody.

@RestControllerAdvice

Centraliza traducción de excepciones de controladores REST.

@Scheduled

Ejecuta un método periódicamente; publish reintenta eventos pendientes del outbox.

@Service

Especializa Component para lógica de aplicación. El rol por sí solo no crea una transacción.

@Setter

Lombok genera setters; un setter no equivale a transición de negocio autorizada.

@Size

Limita longitud o cantidad de elementos del campo anotado.

@SpringBootApplication

Agrupa configuración, auto-configuración y escaneo del paquete de la aplicación. No descubre automáticamente paquetes hermanos como common sin importarlos.

@SpringBootTest

Carga contexto Spring en pruebas; requiere las dependencias configuradas del entorno.

@Table

Indica nombre de tabla y restricciones de mapeo si se declaran.

@Test

Marca un caso de prueba ejecutable por JUnit.

@Transactional

Demarca la transacción local del datasource del servicio. readOnly es orientación de lectura, no un permiso de seguridad. Llamadas internas al mismo objeto no crean automáticamente otro proxy transaccional.

@Valid

Activa validación de campos/anidados del DTO antes de ejecutar la operación.

@Value

Inyecta una propiedad/expresión con valor por defecto si se declara.

@Version

Bloqueo optimista: detecta que otra transacción modificó la fila. En Producto evita sobrescribir inventario con una edición vieja.

6. Modelo de datos y límites de cada base

Las relaciones con FK solo existen dentro de una misma base. Venta.detalle pertenece a Venta, Producto.categoria a Catálogo, Mascota.usuario a Usuario y Profesional.especialidades a Bienestar. Los ids de sede/profesional/venta vistos desde otro servicio son referencias remotas verificadas por HTTP; no hay una gran transacción que englobe las ocho bases.

Venta · HTTP 8081 / PG 15441

ventas, detalle_ventas; Saga y propietario.

• V1__create_venta_tables.sql · crea ventas, detalle_ventas. Ruta: `services/garritas-venta-ms/src/main/resources/db/migration/V1__create_venta_tables.sql`.

• V2__saga_identity.sql · evoluciona estructura/restricciones o añade datos de demostración. Ruta: `services/garritas-venta-ms/src/main/resources/db/migration/V2__saga_identity.sql`.

• V3__transactional_messaging.sql · crea outbox_event, inbox_event. Ruta: `services/garritas-venta-ms/src/main/resources/db/migration/V3__transactional_messaging.sql`.

• V4__sede_venta.sql · evoluciona estructura/restricciones o añade datos de demostración. Ruta: `services/garritas-venta-ms/src/main/resources/db/migration/V4__sede_venta.sql`.

Institucion · HTTP 8082 / PG 15442

sedes; consulta y administración.

• R__sedes_peru.sql · evoluciona estructura/restricciones o añade datos de demostración. Ruta: `services/garritas-institucion-ms/src/main/resources/db/demo/R__sedes_peru.sql`.

• V1__create_institucion_tables.sql · crea sedes. Ruta: `services/garritas-institucion-ms/src/main/resources/db/migration/V1__create_institucion_tables.sql`.

Envio · HTTP 8083 / PG 15443

envios; entrega por venta pagada.

• V1__create_envio_tables.sql · crea envios. Ruta: `services/garritas-envio-ms/src/main/resources/db/migration/V1__create_envio_tables.sql`.

• V2__envio_identity.sql · evoluciona estructura/restricciones o añade datos de demostración. Ruta: `services/garritas-envio-ms/src/main/resources/db/migration/V2__envio_identity.sql`.

Citas · HTTP 8084 / PG 15444

citas; agenda por mascota/profesional/sede.

• V1__create_citas_tables.sql · crea citas. Ruta: `services/garritas-citas-ms/src/main/resources/db/migration/V1__create_citas_tables.sql`.

• V2__citas_identity.sql · evoluciona estructura/restricciones o añade datos de demostración. Ruta: `services/garritas-citas-ms/src/main/resources/db/migration/V2__citas_identity.sql`.

Catalogo · HTTP 8085 / PG 15445

categorias, productos; inventario y reservas.

• R__pet_store_catalog.sql · evoluciona estructura/restricciones o añade datos de demostración. Ruta: `services/garritas-catalogo-ms/src/main/resources/db/demo/R__pet_store_catalog.sql`.

• V1__create_catalogo_tables.sql · crea categorias, productos. Ruta: `services/garritas-catalogo-ms/src/main/resources/db/migration/V1__create_catalogo_tables.sql`.

• V2__inventory_reservations.sql · crea reservas_stock. Ruta: `services/garritas-catalogo-ms/src/main/resources/db/migration/V2__inventory_reservations.sql`.

• V3__transactional_messaging.sql · crea outbox_event, inbox_event. Ruta: `services/garritas-catalogo-ms/src/main/resources/db/migration/V3__transactional_messaging.sql`.

• V4__product_version.sql · evoluciona estructura/restricciones o añade datos de demostración. Ruta: `services/garritas-catalogo-ms/src/main/resources/db/migration/V4__product_version.sql`.

Usuario · HTTP 8086 / PG 15446

usuarios, mascotas; perfil vinculado al sub.

• V1__create_usuario_tables.sql · crea usuarios, mascotas. Ruta: `services/garritas-usuario-ms/src/main/resources/db/migration/V1__create_usuario_tables.sql`.

• V2__profile_identity.sql · evoluciona estructura/restricciones o añade datos de demostración. Ruta: `services/garritas-usuario-ms/src/main/resources/db/migration/V2__profile_identity.sql`.

Pago · HTTP 8087 / PG 15447

pagos; respuesta simulada de Saga.

• V1__create_pago_tables.sql · crea pagos. Ruta: `services/garritas-pago-ms/src/main/resources/db/migration/V1__create_pago_tables.sql`.

• V2__saga_identity.sql · evoluciona estructura/restricciones o añade datos de demostración. Ruta: `services/garritas-pago-ms/src/main/resources/db/migration/V2__saga_identity.sql`.

• V3__transactional_messaging.sql · crea outbox_event, inbox_event. Ruta: `services/garritas-pago-ms/src/main/resources/db/migration/V3__transactional_messaging.sql`.

Bienestar · HTTP 8088 / PG 15448

profesionales, especialidades y unión M:N.

• R__bienestar_demo.sql · evoluciona estructura/restricciones o añade datos de demostración. Ruta: `services/garritas-bienestar-ms/src/main/resources/db/demo/R__bienestar_demo.sql`.

• V1__bienestar.sql · crea especialidades, profesionales, profesional_especialidad. Ruta: `services/garritas-bienestar-ms/src/main/resources/db/migration/V1__bienestar.sql`.



M:N Bienestar: dos FKs locales y referencia de sede remota.

El índice parcial de Citas solo considera PROGRAMADA/CONFIRMADA. Eso permite reusar un bloque tras cancelar, pero no permite dos reservas activas simultáneas. Envío tiene id_venta único incluso si se canceló:

la versión actual no crea un segundo envío para esa misma venta. El perfil usuario admite subject nulo para registros administrativos históricos; para operar como dueño se usa el sub vinculado.

7. Transacciones, concurrencia y dinero

• @Transactional confirma o revierte cambios locales sobre el datasource del módulo. No hace que una llamada HTTP a otra base participe de la misma transacción. Por eso la compra usa eventos y compensación.

• BigDecimal calcula importes y subtotal con precisión decimal. El servidor obtiene el precio vigente de Catálogo y persiste el precio unitario de la compra; no acepta el precio enviado por el carrito como autoridad.

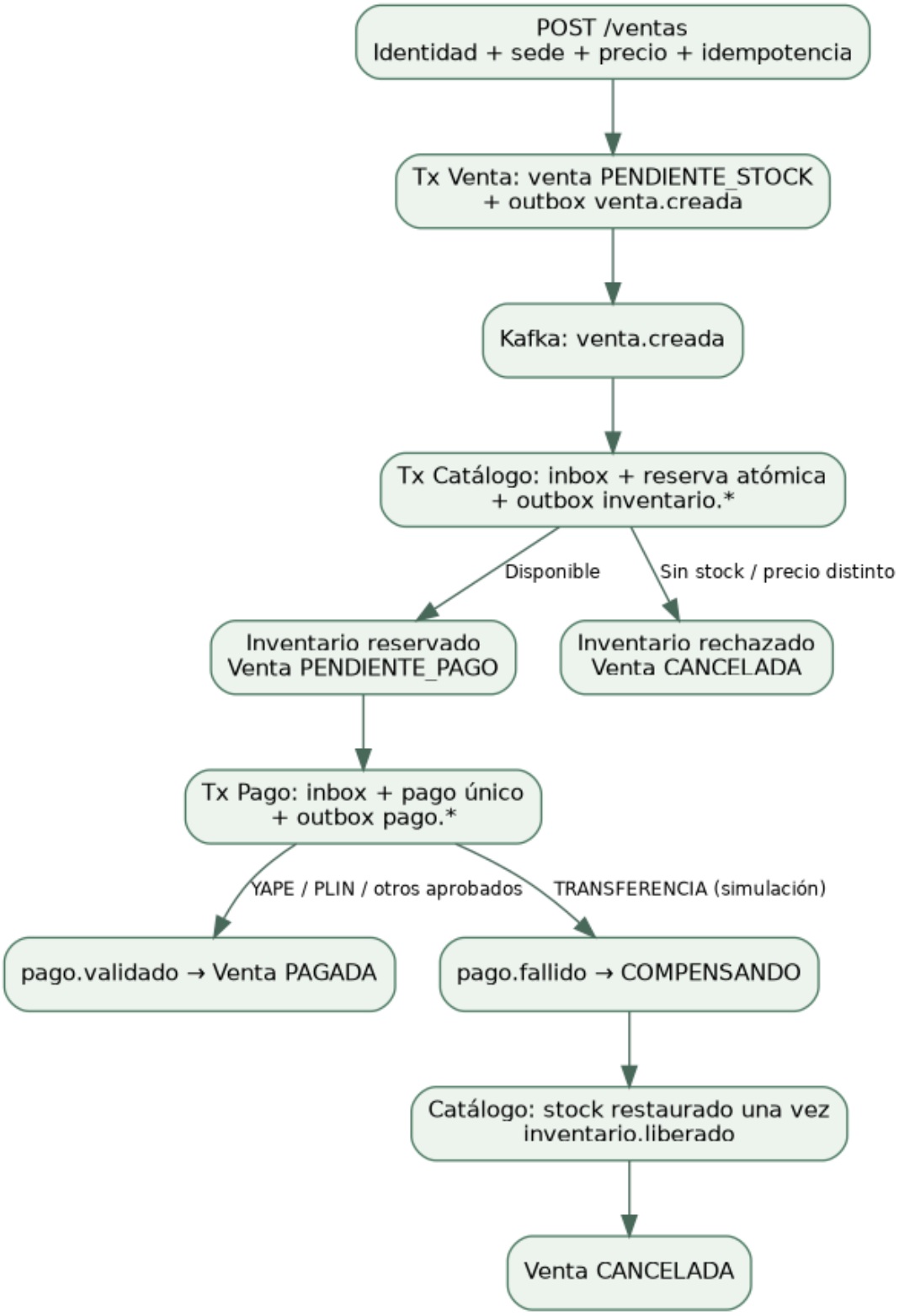
• Catálogo bloquea filas FOR UPDATE en orden de ids para reducir deadlocks y validar todo antes de descontar. Si una línea falla, no descuenta parcialmente el conjunto.

• Producto @Version detecta ediciones obsoletas; la reserva JDBC también incrementa version para no perder cambios frente a una actualización JPA.

• Venta usa advisory lock transaccional por propietario+Idempotency-Key, además de unicidad persistente. Una misma clave reintenta la misma operación; no es un permiso para cambiar su contenido.

• Índices únicos y FKs siguen protegiendo aunque dos peticiones pasen simultáneamente una validación previa. El error de integridad se traduce en conflicto, no se ignora.

8. Compra completa: quién llama a quién



Estados/evidencias de aprobación y rechazo. Las flechas asíncronas pasan por Kafka.

1. CartStore prepara sedeId, metodoPago y detalles de ids/cantidades. Api.crearVenta añade clave UUID. authInterceptor añade Bearer y X-Trace-ID solo a la API propia. Gateway autentica y enruta por Eureka. VentaController convierte JSON, valida y recibe la clave.

2. VentaService determina comprador por sub, comprueba idempotencia, valida sede por ServiceClient y consulta productos por Feign/Circuit Breaker. Calcula totales, guarda venta/detalles y outbox en una transacción. Devuelve la venta inicialmente pendiente; Angular comienza/continúa polling.

3. Outbox publica venta.creada. InventarioEvents deduplica eventId y id_venta, reserva todas las líneas y publica resultado local. VentaEvents toma inventario.reservado y pasa a pendiente de pago; PagoEvents crea una simulación única y publica resultado.

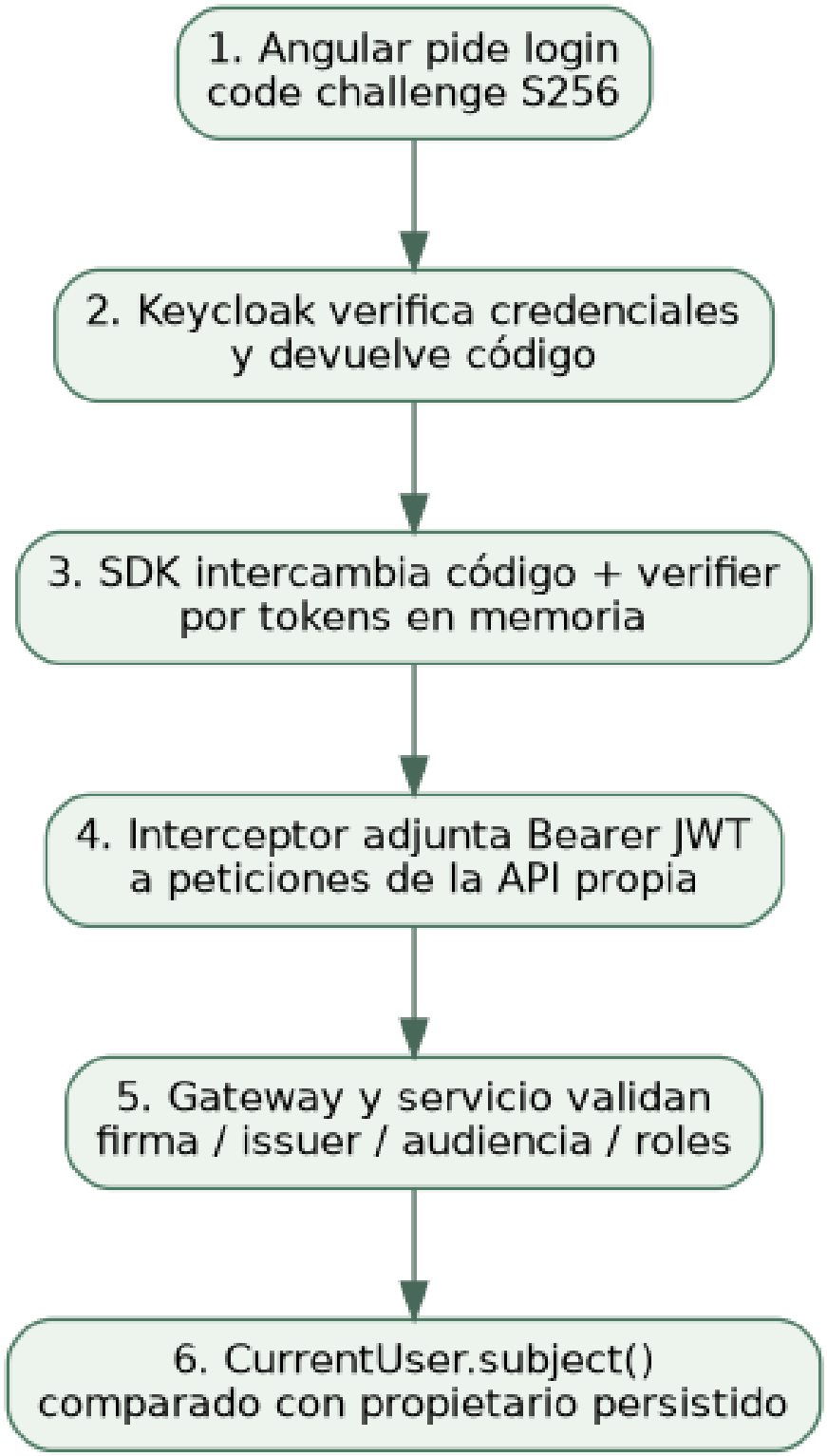
4. Si aprueba, VentaEvents marca PAGADA. Si rechaza, marca COMPENSANDO y Catálogo devuelve las cantidades originales antes de publicar inventario.liberado. Solo entonces la venta se cancela. App actualiza catálogo al terminar pendientes, por lo que el stock visible se refresca.

No hay 2PC, transacción distribuida global ni garantía exactamente una vez. La entrega Kafka es al menos una vez; el procesamiento idempotente combina Inbox y claves de negocio. Un ACK recibido seguido de caída antes de marcar publicado produce reenvío esperado.

9. Fallos, reintentos y límites de resiliencia

Situación	Mecanismo real	Límite
Kafka caído	Outbox conserva published_at nulo y reintenta	La orden permanece pendiente; no se inventa pago
Consumidor recibe duplicado	Inbox eventId + unicidad por venta	La retención/purga debe respetar posibilidad de reenvío
Consumidor falla repetidamente	Nueve reintentos y DLT	DLT requiere análisis; no hay recuperación de negocio automática universal
Catálogo HTTP falla	Timeout y Circuit Breaker; 503	No hay precio fallback ficticio
Cita simultánea	Índice único parcial en PostgreSQL	Solo se certifica el bloque reservado, no un planificador complejo
Cambio de estado terminal	Validación de estado y rol	No hay Saga de devolución/reembolso posterior implementada
Nodo Kafka perdido	Persistencia en volumen, una réplica	No es clúster HA; pérdida del disco no se cubre

10. Seguridad, propiedad y CORS



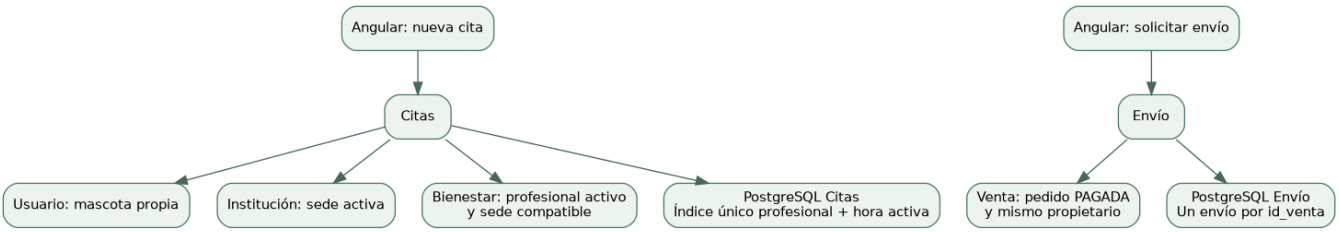
Autenticación del navegador y autorización efectiva del servicio.

Autenticación responde quién es la persona; rol responde qué tipo de acciones puede solicitar; propiedad responde sobre qué registros. CLIENTE autenticado no puede leer la mascota/venta/envío de otro sub. ADMIN puede gestionar y auditar según las rutas. Nunca se confía en un idCliente escrito en el JSON para establecer identidad.

Gateway y servicios validan firma por JWKS, issuer del realm y aud garritas-api. realm_access.roles se convierte a ROLE_ADMIN/ROLE_CLIENTE. Las rutas públicas de productos, categorías, sedes y especialistas permiten consulta; la escritura queda ADMIN. Confirmación/atención de citas y tránsito/entrega de envíos son reglas específicas antes de patrones más amplios.

CORS es política del navegador para permitir solicitudes desde otro origen; no autentica ni bloquea llamadas curl. Se permiten encabezados Authorization, Idempotency-Key y X-Trace-ID para el origen configurado. PKCE protege el intercambio de código del cliente público; no elimina la necesidad de TLS en una publicación real.

11. Citas y envíos conectados al resto



Comprobaciones de negocio antes de guardar los recursos dependientes.

Citas consulta /usuarios/me/mascotas/{id} con el JWT original: la comprobación de dueño la realiza Usuario. Consulta sede y profesional, exige activos y sede compatible si el profesional tiene sede asignada. Guarda los nombres para mostrar historial y su propia referencia/fecha. No escribe la base de Usuario o Bienestar.

Envío consulta /ventas/{id} con identidad propagada; Venta filtra propietario y Envío lo verifica además. PAGADA es requisito y id_venta único. El cliente solo solicita y consulta; el administrador opera tránsito y entrega. No hay geolocalización de transportista ni integración con courier real en esta versión.

12. Angular: desde el arranque a los formularios

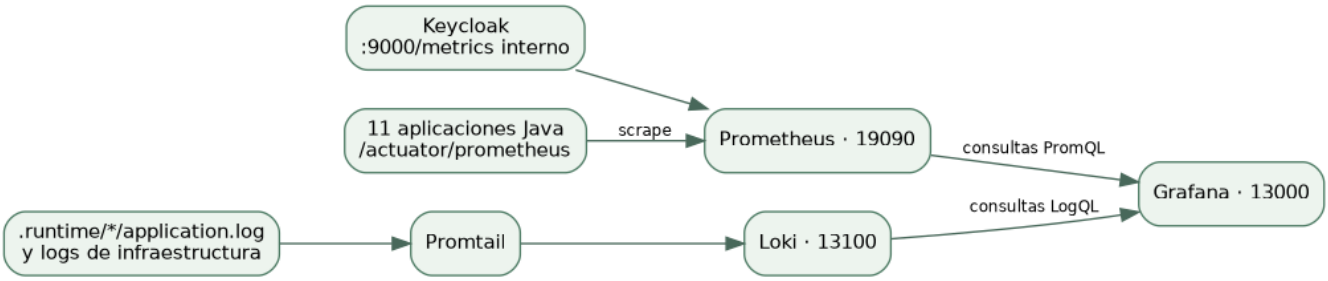
main.ts llama bootstrapApplication(App, appConfig). El inicializador espera Auth.init; Auth espera Config.load. Después de conocer apiUrl/keycloakUrl, Keycloak detecta sesión y actualiza signals. El componente raíz puede decidir menú y cargar catálogo/sedes sin manejar contraseñas.

Signals guardan datos mutables de UI; computed recalcula totales y validez; effect persiste navegación/carrito cuando cambian sus dependencias. RxJS modela HTTP/polling: exhaustMap evita solapar refrescos de pedidos; catchError comunica fallo y takeUntilDestroyed cancela al destruir App. Las interfaces TS documentan forma de datos, no verifican permisos en runtime.

Feedback centraliza error/message/busy para formularios. Cada pantalla usa DomainApi o Api y actualiza su listado después de guardar. Cuenta espera terminar la carga del perfil antes de mostrar inputs: evita que una respuesta tardía reemplace ediciones. La prueba de regresión cubre perfil existente y ausencia 404.

El carrito usa localStorage para contenido no sensible. La navegación usa sessionStorage. Los tokens se mantienen en memoria de keycloak-js. Un badge de rol o un botón oculto son presentación; los filtros Spring y CurrentUser son la barrera efectiva.

13. Observabilidad y correlación



Métricas y logs siguen circuitos diferentes pero se leen desde Grafana.

CorrelationIdFilter añade un identificador al MDC. Feign propaga X-Trace-ID; ServiceClient reenvía identidad y correlación; SagaEvent conserva traceId. Los consumidores vuelven a ponerlo en MDC durante procesamiento. Así una búsqueda de logs puede seguir una operación, aunque no existen spans OpenTelemetry ni una traza temporal completa automática.

Actuator expone readiness/liveness y Prometheus. El grupo readiness configurado no incluye por sí mismo cada remoto: una JVM UP puede devolver 503 de negocio si otra dependencia cae. La verificación real usa catálogo por Gateway, navegador, estado final de venta, stock y permisos. Prometheus no demuestra por sí solo corrección del negocio.

14. Fichas de todas las clases Java

Cada ficha reproduce package/imports/anotaciones y declaración inicial de la clase real; después lista dependencias y firmas detectadas. El cuerpo completo está en el mismo archivo del Manual 2/ZIP. No se imprime aquí por segunda vez todo el código. Los accesores/constructores generados por Lombok no aparecen como métodos escritos.

CurrentUser · garritas-common

Ubicación: `infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/CurrentUser.java`.

Obtiene el sub autenticado y el rol ADMIN desde SecurityContextHolder. requireOwner impide que un cliente acceda a un recurso de otro sub; ADMIN puede auditarlo. No recibe la identidad del JSON del navegador.

```
package pe.edu.upeu.garritas.common;

import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.core.context.SecurityContextHolder;

public final class CurrentUser {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

- `public static String subject()` · Lee el identificador estable del autenticado.
- `public static boolean admin()` · Comprueba la autoridad ROLE_ADMIN.
- `public static void requireOwner(String owner)` · Rechaza el acceso si no es ADMIN y el sub no coincide.

DomainErrors · garritas-common

Ubicación: `infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/DomainErrors.java`.

Traduce excepciones de estado HTTP, validación, integridad y concurrencia a respuestas controladas. Una violación de integridad o conflicto de versión produce 409; no inventa una confirmación de negocio.

```
package pe.edu.upeu.garritas.common;

import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.dao.OptimisticLockingFailureException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.server.ResponseStatusException;

@RestControllerAdvice
public class DomainErrors {
```

Anotaciones presentes en el archivo: @ExceptionHandler, @RestControllerAdvice. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

HttpServices · garritas-common

Ubicación: `infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/HttpServices.java`.

Registra ServiceClient. Construye un RestClient local con límites de conexión/lectura; no publica un RestClient.Builder global balanceado que pueda interferir con Eureka.

```
package pe.edu.upeu.garritas.common;

import java.net.http.HttpClient;
```

```
import java.time.Duration;
import org.springframework.cloud.client.loadbalancer.LoadBalancerClient;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.client.JdkClientHttpRequestFactory;
import org.springframework.web.client.RestClient;

@Configuration(proxyBeanMethods = false)
public class HttpServices {
```

Anotaciones presentes en el archivo: @Bean, @Configuration. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

Inbox · garritas-common

Ubicación: `infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/Inbox.java`.

claim exige transacción activa y registra event_id con ON CONFLICT DO NOTHING. Devuelve false si ya fue procesado. Su inserción y el efecto de negocio se confirman o revierten juntos.

```
package pe.edu.upeu.garritas.common;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.transaction.support.TransactionSynchronizationManager;

public class Inbox {
```

Dependencias inyectadas por constructor: JdbcTemplate jdbc. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public boolean claim(SagaEvent event)` · Registra el eventId dentro de la misma transacción; false significa ya procesado.

MessagingConfiguration · garritas-common

Ubicación: `infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/MessagingConfiguration.java`.

Habilita el planificador y crea Outbox, Inbox, topic de Saga y DLT con tres particiones y réplica 1. Configura nueve reintentos con pausa y publicación en DLT. No proporciona un clúster Kafka de alta disponibilidad.

```
package pe.edu.upeu.garritas.common;

import org.apache.kafka.clients.admin.NewTopic;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.listener.*;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.util.backoff.FixedBackOff;
import tools.jackson.databind.ObjectMapper;

@Configuration(proxyBeanMethods = false)
@EnableScheduling
public class MessagingConfiguration {
```

Anotaciones presentes en el archivo: @Bean, @Configuration, @EnableScheduling, @Value. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

Outbox · garritas-common

Ubicación: `infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/Outbox.java`.

append exige una transacción local activa e inserta el evento en PostgreSQL. publish se programa periódicamente, selecciona pendientes con FOR UPDATE SKIP LOCKED, espera el ACK Kafka y marca published_at. Un fallo conserva el pendiente. Puede duplicar entrega tras un reinicio; Inbox controla ese caso.

```
package pe.edu.upeu.garritas.common;

import java.util.concurrent.TimeUnit;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.transaction.support.TransactionSynchronizationManager;
import tools.jackson.databind.ObjectMapper;

/**
 * DB + outbox commit together. A crash after Kafka ack can duplicate delivery; consumers use Inbox.
 */
public class Outbox {
```

Dependencias inyectadas por constructor: JdbcTemplate jdbc, KafkaTemplate<String, String> kafka, ObjectMapper json, String topic. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @Scheduled, @Transactional, @Value. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public void append(SagaEvent event)` · Guarda evento en outbox dentro de una transacción activa.

• `public void publish()` · Envía pendientes y marca publicado después de ACK, dejando pendientes ante error.

Reference · garritas-common

Ubicación: `infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/Reference.java`.

Record de lectura tolerante a campos JSON adicionales. Representa únicamente los campos remotos necesarios, como id, activo, sedeId, estado e idCliente; no replica entidades ni comparte una base.

```
package pe.edu.upeu.garritas.common;

@com.fasterxml.jackson.annotation.JsonIgnoreProperties(ignoreUnknown = true)
public record Reference(
    Long id, String nombre, Boolean activo, Long sedeId, String estado, String idCliente) {
```

Anotaciones presentes en el archivo: @JsonIgnoreProperties. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public record Reference( Long id, String nombre, Boolean activo, Long sedeId, String estado, String idCliente)` · Operación específica de Reference. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

SagaEvent · garritas-common

Ubicación: `infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/SagaEvent.java`.

Contrato inmutable y versionado del evento de venta; contiene identidad, tipo, items, importes y correlación. validate rechaza contratos inválidos y next produce otro evento con nueva identidad para el mismo agregado.

```
package pe.edu.upeu.garritas.common;

import java.math.BigDecimal;
```

```
import java.time.Instant;
import java.util.List;
import java.util.UUID;

/** Versioned wire contract. No JPA entities or Java type headers cross service boundaries. */
public record SagaEvent(
    UUID eventId,
    int version,
    String type,
    Long ventaId,
    String clienteId,
    BigDecimal total,
    String metodo,
    String traceId,
    List<Item> items,
    Instant occurredAt) {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public record SagaEvent( UUID eventId, int version, String type, Long ventaId, String clienteId, BigDecimal total, String metodo, String traceId, List<Item> items, Instant occurredAt)` · Operación específica de SagaEvent. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public record Item(Long productoId, int cantidad, BigDecimal precio)` · Operación específica de SagaEvent. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public SagaEvent next(String nextType)` · Deriva el siguiente evento conservando contexto de venta y creando identidad de evento.

• `public void validate()` · Verifica invariantes del contrato de evento.

SecurityConfiguration · garritas-common

Ubicación: `infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/SecurityConfiguration.java`.

Declara la cadena de filtros de seguridad, JWT y conversión de roles de realm_access a ROLE_*. Define consultas públicas, rutas del propietario y operaciones exclusivas ADMIN. Se importa tanto en Gateway como en los servicios.

```
package pe.edu.upeu.garritas.common;

import java.util.*;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationConverter;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.web.cors.*;

@Import({HttpServices.class, DomainErrors.class})
@Configuration(proxyBeanMethods = false)
public class SecurityConfiguration {
```

Anotaciones presentes en el archivo: @Bean, @Configuration, @Import, @Value. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

ServiceClient · garritas-common

Ubicación: `infra/garritas-common/src/main/java/pe/edu/upeu/garritas/common/ServiceClient.java`.

Elige una instancia por nombre mediante LoadBalancerClient, compone su URI y reenvía Bearer JWT y X-Trace-ID. Se usa al verificar mascota, sede, profesional o venta. Si no hay instancia o falla la conexión devuelve 503.

```
package pe.edu.upeu.garritas.common;

import org.slf4j.MDC;
import org.springframework.http.HttpStatus;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationToken;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.RestClientException;
import org.springframework.web.server.ResponseStatusException;

/** Bounded, read-only inter-service calls through Eureka, preserving caller identity. */
public class ServiceClient {
```

Dependencias inyectadas por constructor: RestClient client, org.springframework.cloud.client.loadbalancer.LoadBalancerClient loadBalancer. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public <T> T get(String service, String path, Class<T> type)` · Ejecuta la lectura HTTP con la configuración/identidad del cliente declarado.

SagaContractTest · garritas-common

Ubicación: `infra/garritas-common/src/test/java/pe/edu/upeu/garritas/common/SagaContractTest.java`.

Pruebas del contrato versionado, correlación y control de propiedad. Comprueba casos válidos e inválidos sin afirmar entrega Kafka exactamente una vez.

```
package pe.edu.upeu.garritas.common;

import static org.junit.jupiter.api.Assertions.*;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;
import java.util.UUID;
import org.junit.jupiter.api.Test;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;

class SagaContractTest {
```

Anotaciones presentes en el archivo: @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `private SagaEvent event(int version, int cantidad)` · Operación específica de SagaContractTest. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

GarritasConfigApplication · garritas-config

Ubicación:
`infra/garritas-config/src/main/java/pe/edu/upeu/garritas/config/GarritasConfigApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.config;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;
```

```
@SpringBootApplication
@EnableConfigServer
public class GarritasConfigApplication {
```

Anotaciones presentes en el archivo: @EnableConfigServer, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

- `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

GarritasConfigApplicationTests · garritas-config

Ubicación:
`infra/garritas-config/src/test/java/pe/edu/upeu/garritas/config/GarritasConfigApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.config;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class GarritasConfigApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasEurekaApplication · garritas-eureka

Ubicación:
`infra/garritas-eureka/src/main/java/pe/edu/upeu/garritas/eureka/GarritasEurekaApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.eureka;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;

@SpringBootApplication
@EnableEurekaServer
public class GarritasEurekaApplication {
```

Anotaciones presentes en el archivo: @EnableEurekaServer, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

- `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

GarritasEurekaApplicationTests · garritas-eureka

Ubicación:
`infra/garritas-eureka/src/test/java/pe/edu/upeu/garritas/eureka/GarritasEurekaApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.eureka;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class GarritasEurekaApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasGatewayApplication · garritas-gateway

Ubicación:
`infra/garritas-gateway/src/main/java/pe/edu/upeu/garritas/gateway/GarritasGatewayApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.gateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@org.springframework.context.annotation.Import(
    pe.edu.upeu.garritas.common.SecurityConfiguration.class)
@SpringBootApplication
public class GarritasGatewayApplication {
```

Anotaciones presentes en el archivo: @Import, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

- `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

CorrelationIdFilter · garritas-gateway

Ubicación:
`infra/garritas-gateway/src/main/java/pe/edu/upeu/garritas/gateway/filter/CorrelationIdFilter.java`.

Lee o genera X-Trace-ID, lo asocia al MDC del hilo y devuelve la correlación. Permite buscar en los logs una misma operación HTTP; los eventos conservan traceId. No constituye trazado distribuido automático con OpenTelemetry.

```
package pe.edu.upeu.garritas.gateway.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.UUID;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
```

Anotaciones presentes en el archivo: @Component, @Override. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

- `protected void doFilterInternal( HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)` · Operación específica de CorrelationIdFilter. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

GarritasGatewayApplicationTests · garritas-gateway

Ubicación:
`infra/garritas-gateway/src/test/java/pe/edu/upeu/garritas/gateway/GarritasGatewayApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.gateway;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class GarritasGatewayApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasBienestarMsApplication · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/GarritasBienestarMsApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.bienestar;

@org.springframework.boot.autoconfigure.SpringBootApplication
@org.springframework.context.annotation.Import(
    pe.edu.upeu.garritas.common.SecurityConfiguration.class)
public class GarritasBienestarMsApplication {
```

Anotaciones presentes en el archivo: @Import, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

BienestarController · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/controller/BienestarController.java`.

Adaptador REST de Bienestar. Sus mappings declaran rutas, verbo HTTP y respuestas; @Valid activa la validación de entrada. Delega el trabajo en el servicio inyectado. Las restricciones de rol se encuentran además en SecurityConfiguration.

```
package pe.edu.upeu.garritas.bienestar.controller;

import jakarta.validation.Valid;
import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import pe.edu.upeu.garritas.bienestar.dto.*;
import pe.edu.upeu.garritas.bienestar.entity.Especialidad;
import pe.edu.upeu.garritas.bienestar.service.BienestarService;

@RestController
@RequestMapping("/api/v1")
@RequiredArgsConstructor
public class BienestarController {
```

Dependencias inyectadas por constructor: BienestarService service. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping, @RequiredArgsConstructor, @ResponseStatus, @RestController, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<ProfesionalResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public ProfesionalResponse obtener(@PathVariable Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public ProfesionalResponse crear(@Valid @RequestBody ProfesionalRequest r)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public ProfesionalResponse editar( @PathVariable Long id, @Valid @RequestBody ProfesionalRequest r)` · Operación específica de BienestarController. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public void eliminar(@PathVariable Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

• `public List<Especialidad> especialidades()` · Operación específica de BienestarController. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public Especialidad crearEspecialidad(@Valid @RequestBody EspecialidadRequest r)` · Operación específica de BienestarController. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public Especialidad editarEspecialidad( @PathVariable Long id, @Valid @RequestBody EspecialidadRequest r)` · Operación específica de BienestarController. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public void borrarEspecialidad(@PathVariable Long id)` · Operación específica de BienestarController. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

EspecialidadRequest · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/dto/EspecialidadRequest.java`.

DTO de entrada para Especialidad. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.bienestar.dto;

import jakarta.validation.constraints.*;

public record EspecialidadRequest(
    @NotBlank @Size(max = 100) String nombre,
    @Size(max = 255) String descripcion,
    @NotNull Boolean activo) {
```

Anotaciones presentes en el archivo: @NotBlank, @NotNull, @Size. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public record EspecialidadRequest( @NotBlank @Size(max = 100) String nombre, @Size(max = 255) String descripcion, @NotNull Boolean activo)` · Operación específica de EspecialidadRequest. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

ProfesionalRequest · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/dto/ProfesionalRequest.java`.

DTO de entrada para Profesional. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.bienestar.dto;

import jakarta.validation.constraints.*;
import java.util.List;
```

```
public record ProfesionalRequest(
    @NotBlank @Size(max = 100) String nombre,
    @NotBlank @Email @Size(max = 150) String email,
    @Size(max = 20) String telefono,
    @Positive Long sedeId,
    @NotNull Boolean activo,
    @NotEmpty @Size(max = 10) List<@NotNull @Positive Long> especialidadIds) {
```

Anotaciones presentes en el archivo: @Email, @NotBlank, @NotEmpty, @NotNull, @Positive, @Size. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public record ProfesionalRequest( @NotBlank @Size(max = 100) String nombre, @NotBlank @Email @Size(max = 150) String email, @Size(max = 20) String telefono, @Positive Long sedeId, @NotNull Boolean activo, @NotEmpty @Size(max = 10) List<@NotNull @Positive Long> especialidadIds)` · Operación específica de ProfesionalRequest. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

ProfesionalResponse · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/dto/ProfesionalResponse.java`.

DTO de salida para Profesional. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.bienestar.dto;

import java.util.List;
import pe.edu.upeu.garritas.bienestar.entity.Especialidad;

public record ProfesionalResponse(
    Long id,
    String nombre,
    String email,
    String telefono,
    Long sedeId,
    Boolean activo,
    List<Especialidad> especialidades) {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public record ProfesionalResponse( Long id, String nombre, String email, String telefono, Long sedeId, Boolean activo, List<Especialidad> especialidades)` · Operación específica de ProfesionalResponse. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

Especialidad · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/entity/Especialidad.java`.

Entidad JPA Especialidad: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.bienestar.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "especialidades")
@Getter
@Setter
@NoArgsConstructor
public class Especialidad {
```

Anotaciones presentes en el archivo: @Column, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas

corresponden al encabezado de clase.

Campos declarados: Long id, String nombre, String descripcion, Boolean activo. Sus constraints y relaciones están en el código completo.

Profesional · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/entity/Profesional.java`.

Entidad JPA Profesional: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.bienestar.entity;

import jakarta.persistence.*;
import java.util.*;
import lombok.*;

@Entity
@Table(name = "profesionales")
@Getter
@Setter
@NoArgsConstructor
public class Profesional {
```

Anotaciones presentes en el archivo: @Column, @Entity, @GeneratedValue, @Getter, @Id, @JoinColumn, @JoinTable, @ManyToMany, @NoArgsConstructor, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, String nombre, String email, String telefono, Long sedeId, Boolean activo, Set<Especialidad> especialidades. Sus constraints y relaciones están en el código completo.

CorrelationIdFilter · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/filter/CorrelationIdFilter.java`.

Lee o genera X-Trace-ID, lo asocia al MDC del hilo y devuelve la correlación. Permite buscar en los logs una misma operación HTTP; los eventos conservan traceId. No constituye trazado distribuido automático con OpenTelemetry.

```
package pe.edu.upeu.garritas.bienestar.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.UUID;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
```

Anotaciones presentes en el archivo: @Component, @Override. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `protected void doFilterInternal( HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)` · Operación específica de CorrelationIdFilter. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

EspecialidadRepository · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/repository/EspecialidadRepository.java`.

Puerto de persistencia Spring Data de Especialidad. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.bienestar.repository;

public interface EspecialidadRepository
    extends org.springframework.data.jpa.repository.JpaRepository<
        pe.edu.upeu.garritas.bienestar.entity.Especialidad, Long> {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

ProfesionalRepository · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/repository/ProfesionalRepository.java`.

Puerto de persistencia Spring Data de Profesional. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.bienestar.repository;

import java.util.*;
import org.springframework.data.jpa.repository.*;
import pe.edu.upeu.garritas.bienestar.entity.Profesional;

public interface ProfesionalRepository extends JpaRepository<Profesional, Long> {
```

Anotaciones presentes en el archivo: @EntityGraph, @Override. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `List<Profesional> findAll()` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

• `Optional<Profesional> findById(Long id)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

• `long countByEspecialidades_Id(Long id)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

BienestarService · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/service/BienestarService.java`.

Gestiona profesionales y especialidades con transacciones. Valida sede opcional y especialidades activas, evita duplicar ids de asociaciones, actualiza el conjunto y bloquea eliminar una especialidad vinculada. La baja de profesionales es lógica.

```
package pe.edu.upeu.garritas.bienestar.service;

import java.util.*;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.server.ResponseStatusException;
import pe.edu.upeu.garritas.bienestar.dto.*;
import pe.edu.upeu.garritas.bienestar.entity.*;
import pe.edu.upeu.garritas.bienestar.repository.*;
import pe.edu.upeu.garritas.common.*;
```

```
@Service
@RequiredArgsConstructor
@Transactional
public class BienestarService {
```

Dependencias inyectadas por constructor: ProfesionalRepository profesionales, EspecialidadRepository especialidades, ServiceClient services. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<ProfesionalResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public ProfesionalResponse obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public ProfesionalResponse guardar(Long id, ProfesionalRequest r)` · Operación específica de BienestarService. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public void eliminar(Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

• `public List<Especialidad> especialidades()` · Operación específica de BienestarService. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public Especialidad guardarEspecialidad(Long id, EspecialidadRequest r)` · Operación específica de BienestarService. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public void eliminarEspecialidad(Long id)` · Operación específica de BienestarService. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `private Profesional profesional(Long id)` · Operación específica de BienestarService. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `private ProfesionalResponse response(Profesional p)` · Operación específica de BienestarService. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

EspecialidadesIntegrationTests · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/test/java/pe/edu/upeu/garritas/bienestar/EspecialidadesIntegrationTests.java`.

Prueba de integración real de PostgreSQL/JPA: asociaciones muchos a muchos, ids duplicados, especialidad compartida, reasignación y conflicto al borrar una especialidad vinculada. Revierte su transacción de prueba.

```
package pe.edu.upeu.garritas.bienestar;

import static org.junit.jupiter.api.Assertions.*;

import java.util.List;
import java.util.UUID;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.server.ResponseStatusException;
```

```
import pe.edu.upeu.garritas.bienestar.dto.EspecialidadRequest;
import pe.edu.upeu.garritas.bienestar.dto.ProfesionalRequest;
import pe.edu.upeu.garritas.bienestar.repository.ProfesionalRepository;
import pe.edu.upeu.garritas.bienestar.service.BienestarService;

@SpringBootTest
@Transactional
class EspecialidadesIntegrationTests {
```

Anotaciones presentes en el archivo: @AfterEach, @Autowired, @BeforeEach, @SpringBootTest, @Test, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasBienestarMsApplicationTests · garritas-bienestar-ms

Ubicación: `services/garritas-bienestar-ms/src/test/java/pe/edu/upeu/garritas/bienestar/GarritasBienestarMsApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.bienestar;

@org.springframework.boot.test.context.SpringBootTest
class GarritasBienestarMsApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasCatalogoMsApplication · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/GarritasCatalogoMsApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.catalogo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@org.springframework.context.annotation.Import({
  pe.edu.upeu.garritas.common.SecurityConfiguration.class,
  pe.edu.upeu.garritas.common.MessagingConfiguration.class
})
@SpringBootApplication
public class GarritasCatalogoMsApplication {
```

Anotaciones presentes en el archivo: @Import, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

- `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

CategoriaController · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/controller/CategoriaController.java`.

Adaptador REST de Categoria. Sus mappings declaran rutas, verbo HTTP y respuestas; @Valid activa la validación de entrada. Delega el trabajo en el servicio inyectado. Las restricciones de rol se encuentran

además en SecurityConfiguration.

```
package pe.edu.upeu.garritas.catalogo.controller;

import pe.edu.upeu.garritas.catalogo.dto.CategoriaRequest;
import pe.edu.upeu.garritas.catalogo.dto.CategoriaResponse;
import pe.edu.upeu.garritas.catalogo.service.CategoriaService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/categorias")
@RequiredArgsConstructor
public class CategoriaController {
```

Dependencias inyectadas por constructor: CategoriaService categoriaService. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping, @RequiredArgsConstructor, @ResponseStatus, @RestController, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<CategoriaResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public CategoriaResponse obtener(@PathVariable Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public CategoriaResponse crear(@Valid @RequestBody CategoriaRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public CategoriaResponse actualizar(@PathVariable Long id, @Valid @RequestBody CategoriaRequest request)` · Busca el existente y reemplaza campos permitidos dentro del flujo del servicio.

• `public void eliminar(@PathVariable Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

ProductoController · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/controller/ProductoController.java`.

Adaptador REST de Producto. Sus mappings declaran rutas, verbo HTTP y respuestas; @Valid activa la validación de entrada. Delega el trabajo en el servicio inyectado. Las restricciones de rol se encuentran además en SecurityConfiguration.

```
package pe.edu.upeu.garritas.catalogo.controller;

import pe.edu.upeu.garritas.catalogo.dto.ProductoRequest;
import pe.edu.upeu.garritas.catalogo.dto.ProductoResponse;
import pe.edu.upeu.garritas.catalogo.service.ProductoService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/productos")
@RequiredArgsConstructor
public class ProductoController {
```

Dependencias inyectadas por constructor: ProductoService productoService. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping, @RequiredArgsConstructor, @ResponseStatus, @RestController, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<ProductoResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public ProductoResponse obtener(@PathVariable Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public ProductoResponse crear(@Valid @RequestBody ProductoRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public ProductoResponse actualizar(@PathVariable Long id, @Valid @RequestBody ProductoRequest request)` · Busca el existente y reemplaza campos permitidos dentro del flujo del servicio.

• `public void eliminar(@PathVariable Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

CategoriaRequest · garritas-catalogo-ms

Ubicación:
`services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/dto/CategoriaRequest.java`.

DTO de entrada para Categoria. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.catalogo.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class CategoriaRequest {
```

Anotaciones presentes en el archivo: @Getter, @NotBlank, @Setter, @Size. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: String nombre, String descripcion. Sus constraints y relaciones están en el código completo.

CategoriaResponse · garritas-catalogo-ms

Ubicación:
`services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/dto/CategoriaResponse.java`.

DTO de salida para Categoria. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.catalogo.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class CategoriaResponse {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, String nombre, String descripcion. Sus constraints y relaciones están en el código completo.

ProductoRequest · garritas-catalogo-ms

Ubicación:
`services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/dto/ProductoRequest.java`.

DTO de entrada para Producto. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.catalogo.dto;

import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.Getter;
import lombok.Setter;
import java.math.BigDecimal;

@Getter
@Setter
public class ProductoRequest {
```

Anotaciones presentes en el archivo: @DecimalMin, @Getter, @Min, @NotBlank, @NotNull, @Setter, @Size. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: String nombre, String descripcion, BigDecimal precio, Integer stock, Boolean activo, Long categoriaId. Sus constraints y relaciones están en el código completo.

ProductoResponse · garritas-catalogo-ms

Ubicación:
`services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/dto/ProductoResponse.java`.

DTO de salida para Producto. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.catalogo.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import java.math.BigDecimal;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class ProductoResponse {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, String nombre, String descripcion, BigDecimal precio, Integer stock, Boolean activo, CategoriaResponse categoria. Sus constraints y relaciones están en el código completo.

Categoria · garritas-catalogo-ms

Ubicación:
`services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/entity/Categoria.java`.

Entidad JPA Categoria: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.catalogo.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "categorias")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Categoria {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Column, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, String nombre, String descripcion. Sus constraints y relaciones están en el código completo.

Producto · garritas-catalogo-ms

Ubicación:
`services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/entity/Producto.java`.

Entidad JPA Producto: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.catalogo.entity;

import jakarta.persistence.*;
import java.math.BigDecimal;
import lombok.*;

@Entity
@Table(name = "productos")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Producto {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Column, @Entity, @GeneratedValue, @Getter, @Id, @JoinColumn, @ManyToOne, @NoArgsConstructor, @Setter, @Table, @Version. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, String nombre, String descripcion, BigDecimal precio, Integer stock, Boolean activo, Categoria categoria. Sus constraints y relaciones están en el código completo.

GlobalExceptionHandler · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/exception/GlobalExceptionHandler.java`.

Advice REST del módulo: convierte las excepciones de recurso inexistente, validación u otros casos declarados en respuestas HTTP. Se debe leer junto a DomainErrors y a Spring Security; no controla por sí solo todos los fallos del sistema.

```
package pe.edu.upeu.garritas.catalogo.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import io.micrometer.core.instrument.MeterRegistry;
import lombok.RequiredArgsConstructor;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
@RequiredArgsConstructor
public class GlobalExceptionHandler {
```

Dependencias inyectadas por constructor: MeterRegistry meterRegistry. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @ExceptionHandler, @RequiredArgsConstructor, @RestControllerAdvice. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

ResourceNotFoundException · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/exception/ResourceNotFoundException.java`.

Excepción de dominio que indica que una búsqueda no encontró el recurso. El manejador de excepciones correspondiente la traduce a una respuesta, evitando devolver una entidad nula.

```
package pe.edu.upeu.garritas.catalogo.exception;

public class ResourceNotFoundException extends RuntimeException {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

CorrelationIdFilter · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/filter/CorrelationIdFilter.java`.

Lee o genera X-Trace-ID, lo asocia al MDC del hilo y devuelve la correlación. Permite buscar en los logs una misma operación HTTP; los eventos conservan traceId. No constituye trazado distribuido automático con OpenTelemetry.

```
package pe.edu.upeu.garritas.catalogo.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.UUID;
```

```
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
```

Anotaciones presentes en el archivo: @Component, @Override. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `protected void doFilterInternal( HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)` · Operación específica de CorrelationIdFilter. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

CategoriaMapper · garritas-catalogo-ms

Ubicación:
`services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/mapper/CategoriaMapper.java`.

Convierte Categoria entre DTO y entidad; separa el contrato HTTP de JPA y establece los valores iniciales que declara su implementación. No debe sustituir las validaciones del servicio.

```
package pe.edu.upeu.garritas.catalogo.mapper;

import pe.edu.upeu.garritas.catalogo.dto.CategoriaRequest;
import pe.edu.upeu.garritas.catalogo.dto.CategoriaResponse;
import pe.edu.upeu.garritas.catalogo.entity.Categoria;
import org.springframework.stereotype.Component;

@Component
public class CategoriaMapper {
```

Anotaciones presentes en el archivo: @Component. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public Categoria toEntity(CategoriaRequest request)` · Convierte entrada a entidad y asigna los valores iniciales declarados.

• `public CategoriaResponse toResponse(Categoria categoria)` · Proyecta campos de entidad a DTO de salida.

ProductoMapper · garritas-catalogo-ms

Ubicación:
`services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/mapper/ProductoMapper.java`.

Convierte Producto entre DTO y entidad; separa el contrato HTTP de JPA y establece los valores iniciales que declara su implementación. No debe sustituir las validaciones del servicio.

```
package pe.edu.upeu.garritas.catalogo.mapper;

import pe.edu.upeu.garritas.catalogo.dto.ProductoRequest;
import pe.edu.upeu.garritas.catalogo.dto.ProductoResponse;
import pe.edu.upeu.garritas.catalogo.entity.Producto;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
public class ProductoMapper {
```

Dependencias inyectadas por constructor: CategoriaMapper categoriaMapper. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @Component, @RequiredArgsConstructor. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public Producto toEntity(ProductoRequest request)` · Convierte entrada a entidad y asigna los valores iniciales declarados.

• `public ProductoResponse toResponse(Producto producto)` · Proyecta campos de entidad a DTO de salida.

InventarioEvents · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/messaging/InventarioEvents.java`.

Reserva todos los artículos de forma atómica con bloqueos por producto en orden determinista. Verifica disponibilidad, actividad y precio antes de descontar. Deduplica tanto event_id como id_venta. Ante pago.fallido libera una reserva una sola vez usando el payload original persistido.

```
package pe.edu.upeu.garritas.catalogo.messaging;

import java.math.BigDecimal;
import java.util.*;
import lombok.RequiredArgsConstructor;
import org.slf4j.*;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;
import pe.edu.upeu.garritas.common.*;
import tools.jackson.databind.ObjectMapper;

@Component
@RequiredArgsConstructor
public class InventarioEvents {
```

Dependencias inyectadas por constructor: JdbcTemplate jdbc, Inbox inbox, Outbox outbox, ObjectMapper json. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @Component, @KafkaListener, @RequiredArgsConstructor, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public void consume(String payload)` · Punto de entrada Kafka: deserializa contrato, filtra tipo, deduplica y ejecuta el efecto local.

• `private void reservar(SagaEvent e, String payload)` · Valida/bloquea todos los productos antes de aplicar el descuento atómico.

• `private void liberar(SagaEvent e)` · Restaura cantidades desde la reserva original solo cuando sigue reservada.

CategoriaRepository · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/repository/CategoriaRepository.java`.

Puerto de persistencia Spring Data de Categoria. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.catalogo.repository;

import pe.edu.upeu.garritas.catalogo.entity.Categoria;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CategoriaRepository extends JpaRepository<Categoria, Long> {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

ProductoRepository · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/repository/ProductoRepository.java`.

Puerto de persistencia Spring Data de Producto. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.catalogo.repository;

import pe.edu.upeu.garritas.catalogo.entity.Producto;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import java.util.List;

public interface ProductoRepository extends JpaRepository<Producto, Long> {
```

Anotaciones presentes en el archivo: @Query. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `List<Producto> findAllConCategoria()` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

CategoriaService · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/service/CategoriaService.java`.

Lógica de aplicación de Categoria. Coordina repositorios/mappers y las validaciones presentes en sus métodos. Los límites @Transactional gobiernan los cambios locales; ver el listado completo para las operaciones exactas.

```
package pe.edu.upeu.garritas.catalogo.service;

import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import pe.edu.upeu.garritas.catalogo.dto.CategoriaRequest;
import pe.edu.upeu.garritas.catalogo.dto.CategoriaResponse;
import pe.edu.upeu.garritas.catalogo.entity.Categoria;
import pe.edu.upeu.garritas.catalogo.exception.ResourceNotFoundException;
import pe.edu.upeu.garritas.catalogo.mapper.CategoriaMapper;
import pe.edu.upeu.garritas.catalogo.repository.CategoriaRepository;

@org.springframework.transaction.annotation.Transactional
@Service
@RequiredArgsConstructor
public class CategoriaService {
```

Dependencias inyectadas por constructor: CategoriaRepository categoriaRepository, CategoriaMapper categoriaMapper. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<CategoriaResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public CategoriaResponse obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public CategoriaResponse crear(CategoriaRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public CategoriaResponse actualizar(Long id, CategoriaRequest request)` · Busca el existente y reemplaza campos permitidos dentro del flujo del servicio.

• `public void eliminar(Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

• `private Categoria buscarOFallar(Long id)` · Busca o lanza excepción; donde procede también exige propiedad.

ProductoService · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/service/ProductoService.java`.

Lógica de aplicación de Producto. Coordina repositorios/mappers y las validaciones presentes en sus métodos. Los límites @Transactional gobiernan los cambios locales; ver el listado completo para las operaciones exactas.

```
package pe.edu.upeu.garritas.catalogo.service;

import io.micrometer.core.instrument.MeterRegistry;
import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import pe.edu.upeu.garritas.catalogo.dto.ProductoRequest;
import pe.edu.upeu.garritas.catalogo.dto.ProductoResponse;
import pe.edu.upeu.garritas.catalogo.entity.Categoria;
import pe.edu.upeu.garritas.catalogo.entity.Producto;
import pe.edu.upeu.garritas.catalogo.exception.ResourceNotFoundException;
import pe.edu.upeu.garritas.catalogo.mapper.ProductoMapper;
import pe.edu.upeu.garritas.catalogo.repository.CategoriaRepository;
import pe.edu.upeu.garritas.catalogo.repository.ProductoRepository;

@org.springframework.transaction.annotation.Transactional
@Service
@RequiredArgsConstructor
public class ProductoService {
```

Dependencias inyectadas por constructor: ProductoRepository productoRepository, ProductoMapper productoMapper, CategoriaRepository categoriaRepository, MeterRegistry meterRegistry. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<ProductoResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public ProductoResponse obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public ProductoResponse crear(ProductoRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public ProductoResponse actualizar(Long id, ProductoRequest request)` · Busca el existente y reemplaza campos permitidos dentro del flujo del servicio.

• `public void eliminar(Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

• `private Producto buscarOFallar(Long id)` · Busca o lanza excepción; donde procede también exige propiedad.

• `private Categoria buscarCategoriaOFallar(Long categoriaId)` · Operación específica de ProductoService. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

GarritasCatalogoMsApplicationTests · garritas-catalogo-ms

Ubicación: `services/garritas-catalogo-ms/src/test/java/pe/edu/upeu/garritas/catalogo/GarritasCatalogoMsApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.catalogo;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class GarritasCatalogoMsApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasCitasMsApplication · garritas-citas-ms

Ubicación:
`services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/GarritasCitasMsApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.citas;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@org.springframework.context.annotation.Import(
    pe.edu.upeu.garritas.common.SecurityConfiguration.class)
@SpringBootApplication
public class GarritasCitasMsApplication {
```

Anotaciones presentes en el archivo: @Import, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

- `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

CitaController · garritas-citas-ms

Ubicación:
`services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/controller/CitaController.java`.

Adaptador REST de Cita. Sus mappings declaran rutas, verbo HTTP y respuestas; @Valid activa la validación de entrada. Delega el trabajo en el servicio inyectado. Las restricciones de rol se encuentran además en SecurityConfiguration.

```
package pe.edu.upeu.garritas.citas.controller;

import pe.edu.upeu.garritas.citas.dto.CitaRequest;
import pe.edu.upeu.garritas.citas.dto.CitaResponse;
import pe.edu.upeu.garritas.citas.service.CitaService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/citas")
@RequiredArgsConstructor
public class CitaController {
```

Dependencias inyectadas por constructor: CitaService citaService. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @GetMapping, @PathVariable, @PostMapping, @RequestBody, @RequestMapping, @RequiredArgsConstructor, @ResponseStatus, @RestController, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<CitaResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public CitaResponse obtener(@PathVariable Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public CitaResponse crear(@Valid @RequestBody CitaRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public CitaResponse confirmar(@PathVariable Long id)` · Pide transición PROGRAMADA → CONFIRMADA; protegida para ADMIN.

• `public CitaResponse atender(@PathVariable Long id)` · Pide transición CONFIRMADA → ATENDIDA; protegida para ADMIN.

• `public CitaResponse cancelar(@PathVariable Long id)` · Aplica la cancelación permitida por el estado y la propiedad de este dominio.

CitaRequest · garritas-citas-ms

Ubicación: `services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/dto/CitaRequest.java`.

DTO de entrada para Cita. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.citas.dto;

import jakarta.validation.constraints.*;
import java.time.Instant;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class CitaRequest {
```

Anotaciones presentes en el archivo: @Future, @Getter, @NotBlank, @NotNull, @Positive, @Setter, @Size. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: String motivo. Sus constraints y relaciones están en el código completo.

CitaResponse · garritas-citas-ms

Ubicación: `services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/dto/CitaResponse.java`.

DTO de salida para Cita. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.citas.dto;

import java.time.Instant;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import pe.edu.upeu.garritas.citas.entity.EstadoCita;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class CitaResponse {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, Long profesionalId, String nombreMascota, String nombreSede, String nombreProfesional, String idCliente, Long idMascota, Long idSede, Instant fechaHora, String motivo, EstadoCita estado. Sus constraints y relaciones están en el código completo.

Cita · garritas-citas-ms

Ubicación: `services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/entity/Cita.java`.

Entidad JPA Cita: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.citas.entity;

import jakarta.persistence.*;
import java.time.Instant;
import lombok.*;

@Entity
@Table(name = "citas")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Cita {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Column, @Entity, @Enumerated, @GeneratedValue, @Getter, @Id, @NoArgsConstructor, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long profesionalId, String nombreMascota, String nombreSede, String nombreProfesional, Long id, String idCliente, Long idMascota, Long idSede, Instant fechaHora, String motivo, EstadoCita estado. Sus constraints y relaciones están en el código completo.

EstadoCita · garritas-citas-ms

Ubicación: `services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/entity/EstadoCita.java`.

Enumeración de valores admitidos por el dominio. Sus constantes impiden usar cadenas arbitrarias en el estado/método de la entidad; las transiciones válidas se controlan en servicios o consumidores.

```
package pe.edu.upeu.garritas.citas.entity;

public enum EstadoCita {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Los componentes del record o las constantes del enum forman su contrato; consulta el listado completo y la explicación de records/enums.

GlobalExceptionHandler · garritas-citas-ms

Ubicación: `services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/exception/GlobalExceptionHandler.java`.

Advice REST del módulo: convierte las excepciones de recurso inexistente, validación u otros casos declarados en respuestas HTTP. Se debe leer junto a DomainErrors y a Spring Security; no controla por sí solo todos los fallos del sistema.

```
package pe.edu.upeu.garritas.citas.exception;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {
```

Anotaciones presentes en el archivo: @ExceptionHandler, @RestControllerAdvice. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

ResourceNotFoundException · garritas-citas-ms

Ubicación: `services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/exception/ResourceNotFoundException.java`.

Excepción de dominio que indica que una búsqueda no encontró el recurso. El manejador de excepciones correspondiente la traduce a una respuesta, evitando devolver una entidad nula.

```
package pe.edu.upeu.garritas.citas.exception;

public class ResourceNotFoundException extends RuntimeException {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

CorrelationIdFilter · garritas-citas-ms

Ubicación: `services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/filter/CorrelationIdFilter.java`.

Lee o genera X-Trace-ID, lo asocia al MDC del hilo y devuelve la correlación. Permite buscar en los logs una misma operación HTTP; los eventos conservan traceId. No constituye trazado distribuido automático con OpenTelemetry.

```
package pe.edu.upeu.garritas.citas.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.UUID;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
```

Anotaciones presentes en el archivo: @Component, @Override. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `protected void doFilterInternal( HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)` · Operación específica de CorrelationIdFilter. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

CitaMapper · garritas-citas-ms

Ubicación: `services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/mapper/CitaMapper.java`.

Convierte Cita entre DTO y entidad; separa el contrato HTTP de JPA y establece los valores iniciales que declara su implementación. No debe sustituir las validaciones del servicio.

```
package pe.edu.upeu.garritas.citas.mapper;

import org.springframework.stereotype.Component;
import pe.edu.upeu.garritas.citas.dto.CitaRequest;
import pe.edu.upeu.garritas.citas.dto.CitaResponse;
import pe.edu.upeu.garritas.citas.entity.Cita;
import pe.edu.upeu.garritas.citas.entity.EstadoCita;

@Component
public class CitaMapper {
```

Anotaciones presentes en el archivo: @Component. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public Cita toEntity(CitaRequest request)` · Convierte entrada a entidad y asigna los valores iniciales declarados.

• `public CitaResponse toResponse(Cita cita)` · Proyecta campos de entidad a DTO de salida.

CitaRepository · garritas-citas-ms

Ubicación:
`services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/repository/CitaRepository.java`.

Puerto de persistencia Spring Data de Cita. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.citas.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import pe.edu.upeu.garritas.citas.entity.Cita;

public interface CitaRepository extends JpaRepository<Cita, Long> {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `java.util.List<Cita> findByIdCliente(String idCliente)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

• `List<Cita> findByIdCliente(Long idCliente)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

CitaService · garritas-citas-ms

Ubicación: `services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/service/CitaService.java`.

Verifica intervalos de 30 minutos y consulta la mascota del propietario, sede activa y profesional activo/sede compatible. Guarda nombres de referencia como snapshot. Permite confirmar desde PROGRAMADA, atender desde CONFIRMADA y cancelar salvo cita ya atendida; comprueba propiedad.

```
package pe.edu.upeu.garritas.citas.service;

import java.util.*;
import lombok.RequiredArgsConstructor;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.server.ResponseStatusException;
import pe.edu.upeu.garritas.citas.dto.*;
import pe.edu.upeu.garritas.citas.entity.*;
import pe.edu.upeu.garritas.citas.mapper.CitaMapper;
import pe.edu.upeu.garritas.citas.repository.CitaRepository;
import pe.edu.upeu.garritas.common.*;

@Service
@RequiredArgsConstructor
@Transactional
public class CitaService {
```

Dependencias inyectadas por constructor: CitaRepository repository, CitaMapper mapper, ServiceClient services. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<CitaResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public CitaResponse obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public CitaResponse crear(CitaRequest r)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public CitaResponse confirmar(Long id)` · Pide transición PROGRAMADA → CONFIRMADA; protegida para ADMIN.

• `public CitaResponse atender(Long id)` · Pide transición CONFIRMADA → ATENDIDA; protegida para ADMIN.

• `public CitaResponse cancelar(Long id)` · Aplica la cancelación permitida por el estado y la propiedad de este dominio.

• `private CitaResponse transition(Long id, EstadoCita next, EstadoCita expected)` · Comprueba estado anterior esperado antes de asignar el siguiente.

• `private Cita owned(Long id)` · Busca el recurso y compara propietario con sub, permitiendo auditoría ADMIN.

GarritasCitasMsApplicationTests · garritas-citas-ms

Ubicación: `services/garritas-citas-ms/src/test/java/pe/edu/upeu/garritas/citas/GarritasCitasMsApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.citas;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class GarritasCitasMsApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasEnvioMsApplication · garritas-envio-ms

Ubicación:
`services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/GarritasEnvioMsApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.envio;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@org.springframework.context.annotation.Import(
    pe.edu.upeu.garritas.common.SecurityConfiguration.class)
@SpringBootApplication
public class GarritasEnvioMsApplication {
```

Anotaciones presentes en el archivo: @Import, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

EnvioController · garritas-envio-ms

Ubicación:
`services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/controller/EnvioController.java`.

Adaptador REST de Envio. Sus mappings declaran rutas, verbo HTTP y respuestas; @Valid activa la validación de entrada. Delega el trabajo en el servicio inyectado. Las restricciones de rol se encuentran además en SecurityConfiguration.

```
package pe.edu.upeu.garritas.envio.controller;

import pe.edu.upeu.garritas.envio.dto.EnvioRequest;
import pe.edu.upeu.garritas.envio.dto.EnvioResponse;
import pe.edu.upeu.garritas.envio.service.EnvioService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/envios")
@RequiredArgsConstructor
public class EnvioController {
```

Dependencias inyectadas por constructor: EnvioService envioService. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @GetMapping, @PathVariable, @PostMapping, @RequestBody, @RequestMapping, @RequiredArgsConstructor, @ResponseStatus, @RestController, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<EnvioResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public EnvioResponse obtener(@PathVariable Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public EnvioResponse crear(@Valid @RequestBody EnvioRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public EnvioResponse marcarEnTransito(@PathVariable Long id)` · Exige pendiente, cambia estado y registra fecha de envío.

• `public EnvioResponse marcarEntregado(@PathVariable Long id)` · Exige tránsito antes de marcar entregado.

• `public EnvioResponse cancelar(@PathVariable Long id)` · Aplica la cancelación permitida por el estado y la propiedad de este dominio.

EnvioRequest · garritas-envio-ms

Ubicación: `services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/dto/EnvioRequest.java`.

DTO de entrada para Envio. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.envio.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;
import lombok.Getter;
import lombok.Setter;

import java.time.Instant;

@Getter
@Setter
public class EnvioRequest {
```

Anotaciones presentes en el archivo: @Getter, @NotBlank, @NotNull, @Setter, @Size. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long idVenta, String direccionEntrega, String transportista, Instant fechaEntregaEstimada. Sus constraints y relaciones están en el código completo.

EnvioResponse · garritas-envio-ms

Ubicación: `services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/dto/EnvioResponse.java`.

DTO de salida para Envio. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.envio.dto;

import pe.edu.upeu.garritas.envio.entity.EstadoEnvio;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.time.Instant;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class EnvioResponse {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, Long idVenta, String direccionEntrega, String transportista, EstadoEnvio estado, Instant fechaEnvio, Instant fechaEntregaEstimada. Sus constraints y relaciones están en el código completo.

Envio · garritas-envio-ms

Ubicación: `services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/entity/Envio.java`.

Entidad JPA Envio: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.envio.entity;

import jakarta.persistence.*;
import java.time.Instant;
import lombok.*;

@Entity
@Table(name = "envios")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Envio {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Column, @Entity, @Enumerated, @GeneratedValue, @Getter, @Id, @NoArgsConstructor, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: String idCliente, Long id, Long idVenta, String direccionEntrega, String transportista, EstadoEnvio estado, Instant fechaEnvio, Instant fechaEntregaEstimada. Sus constraints y relaciones están en el código completo.

EstadoEnvio · garritas-envio-ms

Ubicación: `services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/entity/EstadoEnvio.java`.

Enumeración de valores admitidos por el dominio. Sus constantes impiden usar cadenas arbitrarias en el estado/método de la entidad; las transiciones válidas se controlan en servicios o consumidores.

```
package pe.edu.upeu.garritas.envio.entity;

public enum EstadoEnvio {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Los componentes del record o las constantes del enum forman su contrato; consulta el listado completo y la explicación de records/enums.

GlobalExceptionHandler · garritas-envio-ms

Ubicación: `services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/exception/GlobalExceptionHandler.java`.

Advice REST del módulo: convierte las excepciones de recurso inexistente, validación u otros casos declarados en respuestas HTTP. Se debe leer junto a DomainErrors y a Spring Security; no controla por sí solo todos los fallos del sistema.

```
package pe.edu.upeu.garritas.envio.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {
```

Anotaciones presentes en el archivo: @ExceptionHandler, @RestControllerAdvice. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para

sus ramas y efectos concretos.

• `public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

ResourceNotFoundException · garritas-envio-ms

Ubicación: `services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/exception/ResourceNotFoundException.java`.

Excepción de dominio que indica que una búsqueda no encontró el recurso. El manejador de excepciones correspondiente la traduce a una respuesta, evitando devolver una entidad nula.

```
package pe.edu.upeu.garritas.envio.exception;

public class ResourceNotFoundException extends RuntimeException {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

CorrelationIdFilter · garritas-envio-ms

Ubicación: `services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/filter/CorrelationIdFilter.java`.

Lee o genera X-Trace-ID, lo asocia al MDC del hilo y devuelve la correlación. Permite buscar en los logs una misma operación HTTP; los eventos conservan traceId. No constituye trazado distribuido automático con OpenTelemetry.

```
package pe.edu.upeu.garritas.envio.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.UUID;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
```

Anotaciones presentes en el archivo: @Component, @Override. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `protected void doFilterInternal( HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)` · Operación específica de CorrelationIdFilter. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

EnvioMapper · garritas-envio-ms

Ubicación: `services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/mapper/EnvioMapper.java`.

Convierte Envio entre DTO y entidad; separa el contrato HTTP de JPA y establece los valores iniciales que declara su implementación. No debe sustituir las validaciones del servicio.

```
package pe.edu.upeu.garritas.envio.mapper;

import pe.edu.upeu.garritas.envio.dto.EnvioRequest;
import pe.edu.upeu.garritas.envio.dto.EnvioResponse;
import pe.edu.upeu.garritas.envio.entity.EstadoEnvio;
```

```
import pe.edu.upeu.garritas.envio.entity.Envio;
import org.springframework.stereotype.Component;

import java.time.Instant;

@Component
public class EnvioMapper {
```

Anotaciones presentes en el archivo: @Component. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public Envio toEntity(EnvioRequest request)` · Convierte entrada a entidad y asigna los valores iniciales declarados.

• `public EnvioResponse toResponse(Envio envio)` · Proyecta campos de entidad a DTO de salida.

EnvioRepository · garritas-envio-ms

Ubicación:
`services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/repository/EnvioRepository.java`.

Puerto de persistencia Spring Data de Envio. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.envio.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import pe.edu.upeu.garritas.envio.entity.Envio;

public interface EnvioRepository extends JpaRepository<Envio, Long> {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `java.util.List<Envio> findByIdCliente(String idCliente)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

• `List<Envio> findByIdVenta(Long idVenta)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

EnvioService · garritas-envio-ms

Ubicación:
`services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/service/EnvioService.java`.

Consulta la venta con identidad propagada; exige PAGADA y propietario válido. Genera un envío por venta, pone Por asignar para clientes y controla PENDIENTE, EN_TRANSITO, ENTREGADO y CANCELADO. La seguridad común reserva tránsito y entrega a ADMIN.

```
package pe.edu.upeu.garritas.envio.service;

import java.util.*;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.server.ResponseStatusException;
import pe.edu.upeu.garritas.common.*;
import pe.edu.upeu.garritas.envio.dto.*;
import pe.edu.upeu.garritas.envio.entity.*;
import pe.edu.upeu.garritas.envio.mapper.EnvioMapper;
import pe.edu.upeu.garritas.envio.repository.EnvioRepository;

@Service
@RequiredArgsConstructor
@Transactional
public class EnvioService {
```

Dependencias inyectadas por constructor: EnvioRepository repository, EnvioMapper mapper, ServiceClient services. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<EnvioResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public EnvioResponse obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public EnvioResponse crear(EnvioRequest r)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public EnvioResponse marcarEnTransito(Long id)` · Exige pendiente, cambia estado y registra fecha de envío.

• `public EnvioResponse marcarEntregado(Long id)` · Exige tránsito antes de marcar entregado.

• `public EnvioResponse cancelar(Long id)` · Aplica la cancelación permitida por el estado y la propiedad de este dominio.

• `private Envio owned(Long id)` · Busca el recurso y compara propietario con sub, permitiendo auditoría ADMIN.

GarritasEnvioMsApplicationTests · garritas-envio-ms

Ubicación: `services/garritas-envio-ms/src/test/java/pe/edu/upeu/garritas/envio/GarritasEnvioMsApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.envio;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class GarritasEnvioMsApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasInstitucionMsApplication · garritas-institucion-ms

Ubicación: `services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/GarritasInstitucionMsApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.institucion;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@org.springframework.context.annotation.Import(
    pe.edu.upeu.garritas.common.SecurityConfiguration.class)
@SpringBootApplication
public class GarritasInstitucionMsApplication {
```

Anotaciones presentes en el archivo: @Import, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

SedeController · garritas-institucion-ms

Ubicación: `services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/controller/SedeController.java`.

Adaptador REST de Sede. Sus mappings declaran rutas, verbo HTTP y respuestas; @Valid activa la validación de entrada. Delega el trabajo en el servicio inyectado. Las restricciones de rol se encuentran además en SecurityConfiguration.

```
package pe.edu.upeu.garritas.institucion.controller;

import pe.edu.upeu.garritas.institucion.dto.SedeRequest;
import pe.edu.upeu.garritas.institucion.dto.SedeResponse;
import pe.edu.upeu.garritas.institucion.service.SedeService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/sedes")
@RequiredArgsConstructor
public class SedeController {
```

Dependencias inyectadas por constructor: SedeService sedeService. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping, @RequiredArgsConstructor, @ResponseStatus, @RestController, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<SedeResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public SedeResponse obtener(@PathVariable Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public SedeResponse crear(@Valid @RequestBody SedeRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public SedeResponse actualizar(@PathVariable Long id, @Valid @RequestBody SedeRequest request)` · Busca el existente y reemplaza campos permitidos dentro del flujo del servicio.

• `public void eliminar(@PathVariable Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

SedeRequest · garritas-institucion-ms

Ubicación: `services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/dto/SedeRequest.java`.

DTO de entrada para Sede. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.institucion.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
```

```
import jakarta.validation.constraints.Size;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class SedeRequest {
```

Anotaciones presentes en el archivo: @Getter, @NotBlank, @NotNull, @Setter, @Size. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: String nombre, String direccion, String ciudad, String telefono, Boolean activo. Sus constraints y relaciones están en el código completo.

SedeResponse · garritas-institucion-ms

Ubicación:
`services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/dto/SedeResponse.java`.

DTO de salida para Sede. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.institucion.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class SedeResponse {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, String nombre, String direccion, String ciudad, String telefono, Boolean activo. Sus constraints y relaciones están en el código completo.

Sede · garritas-institucion-ms

Ubicación:
`services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/entity/Sede.java`.

Entidad JPA Sede: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.institucion.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "sedes")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Sede {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Column, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, String nombre, String direccion, String ciudad, String telefono, Boolean activo. Sus constraints y relaciones están en el código completo.

GlobalExceptionHandler · garritas-institucion-ms

Ubicación: `services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/exception/GlobalExceptionHandler.java`.

Advice REST del módulo: convierte las excepciones de recurso inexistente, validación u otros casos declarados en respuestas HTTP. Se debe leer junto a DomainErrors y a Spring Security; no controla por sí solo todos los fallos del sistema.

```
package pe.edu.upeu.garritas.institucion.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {
```

Anotaciones presentes en el archivo: @ExceptionHandler, @RestControllerAdvice. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

ResourceNotFoundException · garritas-institucion-ms

Ubicación: `services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/exception/ResourceNotFoundException.java`.

Excepción de dominio que indica que una búsqueda no encontró el recurso. El manejador de excepciones correspondiente la traduce a una respuesta, evitando devolver una entidad nula.

```
package pe.edu.upeu.garritas.institucion.exception;

public class ResourceNotFoundException extends RuntimeException {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

CorrelationIdFilter · garritas-institucion-ms

Ubicación: `services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/filter/CorrelationIdFilter.java`.

Lee o genera X-Trace-ID, lo asocia al MDC del hilo y devuelve la correlación. Permite buscar en los logs una misma operación HTTP; los eventos conservan traceId. No constituye trazado distribuido automático con OpenTelemetry.

```
package pe.edu.upeu.garritas.institucion.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.UUID;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
```

Anotaciones presentes en el archivo: @Component, @Override. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `protected void doFilterInternal( HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)` · Operación específica de CorrelationIdFilter. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

SedeMapper · garritas-institucion-ms

Ubicación: `services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/mapper/SedeMapper.java`.

Convierte Sede entre DTO y entidad; separa el contrato HTTP de JPA y establece los valores iniciales que declara su implementación. No debe sustituir las validaciones del servicio.

```
package pe.edu.upeu.garritas.institucion.mapper;

import pe.edu.upeu.garritas.institucion.dto.SedeRequest;
import pe.edu.upeu.garritas.institucion.dto.SedeResponse;
import pe.edu.upeu.garritas.institucion.entity.Sede;
import org.springframework.stereotype.Component;

@Component
public class SedeMapper {
```

Anotaciones presentes en el archivo: @Component. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public Sede toEntity(SedeRequest request)` · Convierte entrada a entidad y asigna los valores iniciales declarados.

• `public SedeResponse toResponse(Sede sede)` · Proyecta campos de entidad a DTO de salida.

SedeRepository · garritas-institucion-ms

Ubicación: `services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/repository/SedeRepository.java`.

Puerto de persistencia Spring Data de Sede. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.institucion.repository;

import pe.edu.upeu.garritas.institucion.entity.Sede;
import org.springframework.data.jpa.repository.JpaRepository;

public interface SedeRepository extends JpaRepository<Sede, Long> {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

SedeService · garritas-institucion-ms

Ubicación: `services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/service/SedeService.java`.

Ordena sedes por ciudad/nombre y filtra las inactivas para consultas no administrativas. Crear/editar está protegido por rol y eliminar desactiva; el historial de pedidos y citas no se borra.

```
package pe.edu.upeu.garritas.institucion.service;

import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import pe.edu.upeu.garritas.institucion.dto.SedeRequest;
import pe.edu.upeu.garritas.institucion.dto.SedeResponse;
import pe.edu.upeu.garritas.institucion.entity.Sede;
import pe.edu.upeu.garritas.institucion.exception.ResourceNotFoundException;
import pe.edu.upeu.garritas.institucion.mapper.SedeMapper;
import pe.edu.upeu.garritas.institucion.repository.SedeRepository;

@org.springframework.transaction.annotation.Transactional
@Service
@RequiredArgsConstructor
public class SedeService {
```

Dependencias inyectadas por constructor: SedeRepository sedeRepository, SedeMapper sedeMapper. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<SedeResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public SedeResponse obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public SedeResponse crear(SedeRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public SedeResponse actualizar(Long id, SedeRequest request)` · Busca el existente y reemplaza campos permitidos dentro del flujo del servicio.

• `public void eliminar(Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

• `private Sede buscarOFallar(Long id)` · Busca o lanza excepción; donde procede también exige propiedad.

GarritasInstitucionMsApplicationTests · garritas-institucion-ms

Ubicación: `services/garritas-institucion-ms/src/test/java/pe/edu/upeu/garritas/institucion/GarritasInstitucionMsApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.institucion;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class GarritasInstitucionMsApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasPagoMsApplication · garritas-pago-ms

Ubicación:
`services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/GarritasPagoMsApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.pago;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@org.springframework.context.annotation.Import({
  pe.edu.upeu.garritas.common.SecurityConfiguration.class,
  pe.edu.upeu.garritas.common.MessagingConfiguration.class
})
@SpringBootApplication
public class GarritasPagoMsApplication {
```

Anotaciones presentes en el archivo: @Import, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

PagoController · garritas-pago-ms

Ubicación:
`services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/controller/PagoController.java`.

Adaptador REST de Pago. Sus mappings declaran rutas, verbo HTTP y respuestas; @Valid activa la validación de entrada. Delega el trabajo en el servicio inyectado. Las restricciones de rol se encuentran además en SecurityConfiguration.

```
package pe.edu.upeu.garritas.pago.controller;

import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;
import pe.edu.upeu.garritas.pago.dto.PagoResponse;
import pe.edu.upeu.garritas.pago.service.PagoService;

@RestController
@RequestMapping("/api/v1/pagos")
@RequiredArgsConstructor
public class PagoController {
```

Dependencias inyectadas por constructor: PagoService service. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @GetMapping, @PathVariable, @RequestMapping, @RequiredArgsConstructor, @RestController. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<PagoResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public PagoResponse obtener(@PathVariable Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public PagoResponse porVenta(@PathVariable Long id)` · Busca pago por id de venta y comprueba propietario; puede no existir aún en una Saga pendiente.

PagoRequest · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/dto/PagoRequest.java`.

DTO conservado en el módulo de Pago. En esta versión el controlador no expone una creación libre de pagos: los genera PagoEvents desde inventario.reservado. Su presencia no implica un endpoint de cobro ni una pasarela bancaria.

```
package pe.edu.upeu.garritas.pago.dto;

import pe.edu.upeu.garritas.pago.entity.MetodoPago;
import jakarta.validation.constraints.DecimalMin;
import jakarta.validation.constraints.NotNull;
import lombok.Getter;
import lombok.Setter;

import java.math.BigDecimal;

@Getter
@Setter
public class PagoRequest {
```

Anotaciones presentes en el archivo: @DecimalMin, @Getter, @NotNull, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long idVenta, BigDecimal monto, MetodoPago metodo. Sus constraints y relaciones están en el código completo.

PagoResponse · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/dto/PagoResponse.java`.

DTO de salida para Pago. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.pago.dto;

import pe.edu.upeu.garritas.pago.entity.EstadoPago;
import pe.edu.upeu.garritas.pago.entity.MetodoPago;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

import java.math.BigDecimal;
import java.time.Instant;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class PagoResponse {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, Long idVenta, BigDecimal monto, MetodoPago metodo, EstadoPago estado, Instant fechaPago, String referenciaExterna. Sus constraints y relaciones están en el código completo.

EstadoPago · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/entity/EstadoPago.java`.

Enumeración de valores admitidos por el dominio. Sus constantes impiden usar cadenas arbitrarias en el estado/método de la entidad; las transiciones válidas se controlan en servicios o consumidores.

```
package pe.edu.upeu.garritas.pago.entity;

public enum EstadoPago {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Los componentes del record o las constantes del enum forman su contrato; consulta el listado completo y la explicación de records/enums.

MetodoPago · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/entity/MetodoPago.java`.

Enumeración de valores admitidos por el dominio. Sus constantes impiden usar cadenas arbitrarias en el estado/método de la entidad; las transiciones válidas se controlan en servicios o consumidores.

```
package pe.edu.upeu.garritas.pago.entity;

public enum MetodoPago {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Los componentes del record o las constantes del enum forman su contrato; consulta el listado completo y la explicación de records/enums.

Pago · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/entity/Pago.java`.

Entidad JPA Pago: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.pago.entity;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.Instant;
import lombok.*;

@Entity
@Table(name = "pagos")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Pago {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Column, @Entity, @Enumerated, @GeneratedValue, @Getter, @Id, @NoArgsConstructor, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: String idCliente, Long id, Long idVenta, BigDecimal monto, MetodoPago metodo, EstadoPago estado, Instant fechaPago, String referenciaExterna. Sus constraints y relaciones están en el código completo.

GlobalExceptionHandler · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/exception/GlobalExceptionHandler.java`.

Advice REST del módulo: convierte las excepciones de recurso inexistente, validación u otros casos declarados en respuestas HTTP. Se debe leer junto a DomainErrors y a Spring Security; no controla por sí solo todos los fallos del sistema.

```
package pe.edu.upeu.garritas.pago.exception;
```

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {
```

Anotaciones presentes en el archivo: @ExceptionHandler, @RestControllerAdvice. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

ResourceNotFoundException · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/exception/ResourceNotFoundException.java`.

Excepción de dominio que indica que una búsqueda no encontró el recurso. El manejador de excepciones correspondiente la traduce a una respuesta, evitando devolver una entidad nula.

```
package pe.edu.upeu.garritas.pago.exception;

public class ResourceNotFoundException extends RuntimeException {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

CorrelationIdFilter · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/filter/CorrelationIdFilter.java`.

Lee o genera X-Trace-ID, lo asocia al MDC del hilo y devuelve la correlación. Permite buscar en los logs una misma operación HTTP; los eventos conservan traceId. No constituye trazado distribuido automático con OpenTelemetry.

```
package pe.edu.upeu.garritas.pago.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.UUID;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
```

Anotaciones presentes en el archivo: @Component, @Override. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `protected void doFilterInternal( HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)` · Operación específica de CorrelationIdFilter. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

PagoMapper · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/mapper/PagoMapper.java`.

Convierte Pago entre DTO y entidad; separa el contrato HTTP de JPA y establece los valores iniciales que declara su implementación. No debe sustituir las validaciones del servicio.

```
package pe.edu.upeu.garritas.pago.mapper;

import pe.edu.upeu.garritas.pago.dto.PagoRequest;
import pe.edu.upeu.garritas.pago.dto.PagoResponse;
import pe.edu.upeu.garritas.pago.entity.EstadoPago;
import pe.edu.upeu.garritas.pago.entity.Pago;
import org.springframework.stereotype.Component;

import java.time.Instant;

@Component
public class PagoMapper {
```

Anotaciones presentes en el archivo: @Component. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public Pago toEntity(PagoRequest request)` · Convierte entrada a entidad y asigna los valores iniciales declarados.

• `public PagoResponse toResponse(Pago pago)` · Proyecta campos de entidad a DTO de salida.

PagoEvents · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/messaging/PagoEvents.java`.

Escucha inventario.reservado y crea un pago simulado único por venta dentro de una transacción con Inbox y Outbox. Publica pago.validado o pago.fallido. La regla didáctica configurada rechaza TRANSFERENCIA; no llama a una pasarela bancaria.

```
package pe.edu.upeu.garritas.pago.messaging;

import java.util.*;
import lombok.RequiredArgsConstructor;
import org.slf4j.*;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;
import pe.edu.upeu.garritas.common.*;
import tools.jackson.databind.ObjectMapper;

@Component
@RequiredArgsConstructor
public class PagoEvents {
```

Dependencias inyectadas por constructor: JdbcTemplate jdbc, Inbox inbox, Outbox outbox, ObjectMapper json. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @Component, @KafkaListener, @RequiredArgsConstructor, @Transactional, @Value. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public void consume(String payload)` · Punto de entrada Kafka: deserializa contrato, filtra tipo, deduplica y ejecuta el efecto local.

PagoRepository · garritas-pago-ms

Ubicación:
`services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/repository/PagoRepository.java`.

Puerto de persistencia Spring Data de Pago. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.pago.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import pe.edu.upeu.garritas.pago.entity.Pago;

public interface PagoRepository extends JpaRepository<Pago, Long> {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `List<Pago> findByIdVenta(Long idVenta)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

• `List<Pago> findByIdCliente(String idCliente)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

PagoService · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/service/PagoService.java`.

Lógica de aplicación de Pago. Coordina repositorios/mappers y las validaciones presentes en sus métodos. Los límites @Transactional gobiernan los cambios locales; ver el listado completo para las operaciones exactas.

```
package pe.edu.upeu.garritas.pago.service;

import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.server.ResponseStatusException;
import pe.edu.upeu.garritas.common.CurrentUser;
import pe.edu.upeu.garritas.pago.dto.PagoResponse;
import pe.edu.upeu.garritas.pago.mapper.PagoMapper;
import pe.edu.upeu.garritas.pago.repository.PagoRepository;

@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class PagoService {
```

Dependencias inyectadas por constructor: PagoRepository repository, PagoMapper mapper. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<PagoResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public PagoResponse obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public PagoResponse porVenta(Long id)` · Busca pago por id de venta y comprueba propietario; puede no existir aún en una Saga pendiente.

GarritasPagoMsApplicationTests · garritas-pago-ms

Ubicación: `services/garritas-pago-ms/src/test/java/pe/edu/upeu/garritas/pago/GarritasPagoMsApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.pago;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class GarritasPagoMsApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasUsuarioMsApplication · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/GarritasUsuarioMsApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.usuario;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@org.springframework.context.annotation.Import(
    pe.edu.upeu.garritas.common.SecurityConfiguration.class)
@SpringBootApplication
public class GarritasUsuarioMsApplication {
```

Anotaciones presentes en el archivo: @Import, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

- `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

MascotaController · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/MascotaController.java`.

Adaptador REST de Mascota. Sus mappings declaran rutas, verbo HTTP y respuestas; @Valid activa la validación de entrada. Delega el trabajo en el servicio inyectado. Las restricciones de rol se encuentran además en SecurityConfiguration.

```
package pe.edu.upeu.garritas.usuario.controller;

import jakarta.validation.Valid;
import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import pe.edu.upeu.garritas.usuario.dto.MascotaRequest;
import pe.edu.upeu.garritas.usuario.dto.MascotaResponse;
import pe.edu.upeu.garritas.usuario.service.MascotaService;

@RestController
@RequestMapping("/api/v1/usuarios/{usuarioId}/mascotas")
@RequiredArgsConstructor
public class MascotaController {
```

Dependencias inyectadas por constructor: MascotaService mascotaService. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping, @RequiredArgsConstructor, @ResponseStatus, @RestController, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<MascotaResponse> listar(@PathVariable Long usuarioId)` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public MascotaResponse crear( @PathVariable Long usuarioId, @Valid @RequestBody MascotaRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public MascotaResponse actualizar( @PathVariable Long usuarioId, @PathVariable Long id, @Valid @RequestBody MascotaRequest request)` · Busca el existente y reemplaza campos permitidos dentro del flujo del servicio.

• `public void eliminar(@PathVariable Long usuarioId, @PathVariable Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

MiCuentaController · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/MiCuentaController.java`.

Expone /api/v1/usuarios/me y sus mascotas para el cliente autenticado. Valida DTO y delega en UsuarioService/MascotaService sin admitir un sub arbitrario enviado por el navegador.

```
package pe.edu.upeu.garritas.usuario.controller;

import jakarta.validation.Valid;
import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import pe.edu.upeu.garritas.usuario.dto.*;
import pe.edu.upeu.garritas.usuario.service.*;

@RestController
@RequestMapping("/api/v1/usuarios/me")
@RequiredArgsConstructor
public class MiCuentaController {
```

Dependencias inyectadas por constructor: UsuarioService usuarios, MascotaService mascotas. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping, @RequiredArgsConstructor, @ResponseStatus, @RestController, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public UsuarioResponse me()` · Devuelve el perfil vinculado al sub autenticado.

• `public UsuarioResponse guardar(@Valid @RequestBody UsuarioRequest body)` · Operación específica de MiCuentaController. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public List<MascotaResponse> mascotas()` · Operación específica de MiCuentaController. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public MascotaResponse mascota(@PathVariable Long id)` · Operación específica de MiCuentaController. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public MascotaResponse crear(@Valid @RequestBody MascotaRequest body)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public MascotaResponse editar(@PathVariable Long id, @Valid @RequestBody MascotaRequest body)` · Operación específica de MiCuentaController. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public void eliminar(@PathVariable Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

UsuarioController · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/UsuarioController.java`.

Adaptador REST de Usuario. Sus mappings declaran rutas, verbo HTTP y respuestas; @Valid activa la validación de entrada. Delega el trabajo en el servicio inyectado. Las restricciones de rol se encuentran además en SecurityConfiguration.

```
package pe.edu.upeu.garritas.usuario.controller;

import pe.edu.upeu.garritas.usuario.dto.UsuarioRequest;
import pe.edu.upeu.garritas.usuario.dto.UsuarioResponse;
import pe.edu.upeu.garritas.usuario.service.UsuarioService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/usuarios")
@RequiredArgsConstructor
public class UsuarioController {
```

Dependencias inyectadas por constructor: UsuarioService usuarioService. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @DeleteMapping, @GetMapping, @PathVariable, @PostMapping, @PutMapping, @RequestBody, @RequestMapping, @RequiredArgsConstructor, @ResponseStatus, @RestController, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<UsuarioResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public UsuarioResponse obtener(@PathVariable Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public UsuarioResponse crear(@Valid @RequestBody UsuarioRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public UsuarioResponse actualizar(@PathVariable Long id, @Valid @RequestBody UsuarioRequest request)` · Busca el existente y reemplaza campos permitidos dentro del flujo del servicio.

• `public void eliminar(@PathVariable Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

MascotaRequest · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/dto/MascotaRequest.java`.

DTO de entrada para Mascota. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.usuario.dto;
```

```
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class MascotaRequest {
```

Anotaciones presentes en el archivo: @Getter, @Min, @NotBlank, @Setter, @Size. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: String nombre, String especie, String raza, Integer edad. Sus constraints y relaciones están en el código completo.

MascotaResponse · garritas-usuario-ms

Ubicación:
`services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/dto/MascotaResponse.java`.

DTO de salida para Mascota. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.usuario.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class MascotaResponse {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, String nombre, String especie, String raza, Integer edad. Sus constraints y relaciones están en el código completo.

UsuarioRequest · garritas-usuario-ms

Ubicación:
`services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/dto/UsuarioRequest.java`.

DTO de entrada para Usuario. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.usuario.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class UsuarioRequest {
```

Anotaciones presentes en el archivo: @Email, @Getter, @NotBlank, @Setter, @Size. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: String nombre, String email, String telefono, String direccion. Sus constraints y relaciones están en el código completo.

UsuarioResponse · garritas-usuario-ms

Ubicación:
`services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/dto/UsuarioResponse.java`.

DTO de salida para Usuario. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.usuario.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import java.util.List;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class UsuarioResponse {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, String nombre, String email, String telefono, String direccion, List<MascotaResponse> mascotas. Sus constraints y relaciones están en el código completo.

Mascota · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/entity/Mascota.java`.

Entidad JPA Mascota: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.usuario.entity;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Table(name = "mascotas")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Mascota {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Column, @Entity, @GeneratedValue, @Getter, @Id, @JoinColumn, @ManyToOne, @NoArgsConstructor, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, Usuario usuario, String nombre, String especie, String raza, Integer edad. Sus constraints y relaciones están en el código completo.

Usuario · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/entity/Usuario.java`.

Entidad JPA Usuario: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.usuario.entity;

import jakarta.persistence.*;
```

```
import java.util.ArrayList;
import java.util.List;
import lombok.*;

@Entity
@Table(name = "usuarios")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Usuario {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Column, @Entity, @GeneratedValue, @Getter, @Id, @NoArgsConstructor, @OneToMany, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: String subject, Long id, String nombre, String email, String telefono, String direccion, List<Mascota> mascotas. Sus constraints y relaciones están en el código completo.

GlobalExceptionHandler · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/exception/GlobalExceptionHandler.java`.

Advice REST del módulo: convierte las excepciones de recurso inexistente, validación u otros casos declarados en respuestas HTTP. Se debe leer junto a DomainErrors y a Spring Security; no controla por sí solo todos los fallos del sistema.

```
package pe.edu.upeu.garritas.usuario.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {
```

Anotaciones presentes en el archivo: @ExceptionHandler, @RestControllerAdvice. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

ResourceNotFoundException · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/exception/ResourceNotFoundException.java`.

Excepción de dominio que indica que una búsqueda no encontró el recurso. El manejador de excepciones correspondiente la traduce a una respuesta, evitando devolver una entidad nula.

```
package pe.edu.upeu.garritas.usuario.exception;

public class ResourceNotFoundException extends RuntimeException {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

CorrelationIdFilter · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/filter/CorrelationIdFilter.java`.

Lee o genera X-Trace-ID, lo asocia al MDC del hilo y devuelve la correlación. Permite buscar en los logs una misma operación HTTP; los eventos conservan traceId. No constituye trazado distribuido automático con OpenTelemetry.

```
package pe.edu.upeu.garritas.usuario.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.UUID;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
```

Anotaciones presentes en el archivo: @Component, @Override. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `protected void doFilterInternal( HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)` · Operación específica de CorrelationIdFilter. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

MascotaMapper · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/mapper/MascotaMapper.java`.

Convierte Mascota entre DTO y entidad; separa el contrato HTTP de JPA y establece los valores iniciales que declara su implementación. No debe sustituir las validaciones del servicio.

```
package pe.edu.upeu.garritas.usuario.mapper;

import pe.edu.upeu.garritas.usuario.dto.MascotaRequest;
import pe.edu.upeu.garritas.usuario.dto.MascotaResponse;
import pe.edu.upeu.garritas.usuario.entity.Mascota;
import org.springframework.stereotype.Component;

@Component
public class MascotaMapper {
```

Anotaciones presentes en el archivo: @Component. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public Mascota toEntity(MascotaRequest request)` · Convierte entrada a entidad y asigna los valores iniciales declarados.

• `public MascotaResponse toResponse(Mascota mascota)` · Proyecta campos de entidad a DTO de salida.

UsuarioMapper · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/mapper/UsuarioMapper.java`.

Convierte Usuario entre DTO y entidad; separa el contrato HTTP de JPA y establece los valores iniciales que declara su implementación. No debe sustituir las validaciones del servicio.

```
package pe.edu.upeu.garritas.usuario.mapper;

import pe.edu.upeu.garritas.usuario.dto.UsuarioRequest;
import pe.edu.upeu.garritas.usuario.dto.UsuarioResponse;
import pe.edu.upeu.garritas.usuario.entity.Usuario;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;

@Component
@RequiredArgsConstructor
public class UsuarioMapper {
```

Dependencias inyectadas por constructor: MascotaMapper mascotaMapper. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @Component, @RequiredArgsConstructor. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public Usuario toEntity(UsuarioRequest request)` · Convierte entrada a entidad y asigna los valores iniciales declarados.

• `public UsuarioResponse toResponse(Usuario usuario)` · Proyecta campos de entidad a DTO de salida.

MascotaRepository · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/repository/MascotaRepository.java`.

Puerto de persistencia Spring Data de Mascota. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.usuario.repository;

import pe.edu.upeu.garritas.usuario.entity.Mascota;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface MascotaRepository extends JpaRepository<Mascota, Long> {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `List<Mascota> findByUsuarioId(Long usuarioId)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

UsuarioRepository · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/repository/UsuarioRepository.java`.

Puerto de persistencia Spring Data de Usuario. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.usuario.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import pe.edu.upeu.garritas.usuario.entity.Usuario;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `java.util.Optional<Usuario> findBySubject(String subject)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

MascotaService · garritas-usuario-ms

Ubicación:
`services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/service/MascotaService.java`.

Gestiona mascotas verificando el dueño del perfil asociado. Las rutas propias y las rutas anidadas validan la relación; cambiar el número de una URL no permite ver una mascota de otro cliente.

```
package pe.edu.upeu.garritas.usuario.service;

import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import pe.edu.upeu.garritas.usuario.dto.MascotaRequest;
import pe.edu.upeu.garritas.usuario.dto.MascotaResponse;
import pe.edu.upeu.garritas.usuario.entity.Mascota;
import pe.edu.upeu.garritas.usuario.exception.ResourceNotFoundException;
import pe.edu.upeu.garritas.usuario.mapper.MascotaMapper;
import pe.edu.upeu.garritas.usuario.repository.MascotaRepository;

@org.springframework.transaction.annotation.Transactional
@Service
@RequiredArgsConstructor
public class MascotaService {
```

Dependencias inyectadas por constructor: MascotaRepository mascotaRepository, MascotaMapper mascotaMapper, UsuarioService usuarioService. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<MascotaResponse> listarPorUsuario(Long usuarioId)` · Operación específica de MascotaService. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public MascotaResponse crear(Long usuarioId, MascotaRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public MascotaResponse actualizar(Long id, MascotaRequest request)` · Busca el existente y reemplaza campos permitidos dentro del flujo del servicio.

• `public void eliminar(Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

• `public MascotaResponse obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public void verificarUsuario(Long usuarioId, Long mascotaId)` · Operación específica de MascotaService. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `private Mascota buscarOFallar(Long id)` · Busca o lanza excepción; donde procede también exige propiedad.

UsuarioService · garritas-usuario-ms

Ubicación:
`services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/service/UsuarioService.java`.

Gestiona perfiles de usuario y relaciona el sub de Keycloak con un registro local. La ruta me recupera/actualiza el perfil del autenticado; la gestión general queda para ADMIN. El id numérico de negocio no sustituye al sub para autorizar.

```
package pe.edu.upeu.garritas.usuario.service;

import java.util.List;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import pe.edu.upeu.garritas.usuario.dto.UsuarioRequest;
import pe.edu.upeu.garritas.usuario.dto.UsuarioResponse;
```

```
import pe.edu.upeu.garritas.usuario.entity.Usuario;
import pe.edu.upeu.garritas.usuario.exception.ResourceNotFoundException;
import pe.edu.upeu.garritas.usuario.mapper.UsuarioMapper;
import pe.edu.upeu.garritas.usuario.repository.UsuarioRepository;

@org.springframework.transaction.annotation.Transactional
@Service
@RequiredArgsConstructor
public class UsuarioService {
```

Dependencias inyectadas por constructor: UsuarioRepository usuarioRepository, UsuarioMapper usuarioMapper. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<UsuarioResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public UsuarioResponse obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public UsuarioResponse crear(UsuarioRequest request)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `public UsuarioResponse actualizar(Long id, UsuarioRequest request)` · Busca el existente y reemplaza campos permitidos dentro del flujo del servicio.

• `public void eliminar(Long id)` · Aplica la baja definida: lógica o física según clase; no asumir que DELETE siempre borra filas.

• `public UsuarioResponse me()` · Devuelve el perfil vinculado al sub autenticado.

• `public UsuarioResponse guardarMe(UsuarioRequest request)` · Crea/actualiza perfil propio sin aceptar otro sub en el cuerpo.

• `public Usuario miUsuario()` · Resuelve el perfil local del autenticado; 404 pide completar perfil.

GarritasUsuarioMsApplicationTests · garritas-usuario-ms

Ubicación: `services/garritas-usuario-ms/src/test/java/pe/edu/upeu/garritas/usuario/GarritasUsuarioMsApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.usuario;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class GarritasUsuarioMsApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

GarritasVentaMsApplication · garritas-venta-ms

Ubicación: `services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/GarritasVentaMsApplication.java`.

Punto de entrada Spring Boot del módulo. main llama a SpringApplication.run; sus anotaciones activan el escaneo/configuración del contexto y las importaciones muestran los componentes comunes que utiliza.

```
package pe.edu.upeu.garritas.venta;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@org.springframework.context.annotation.Import({
  pe.edu.upeu.garritas.common.SecurityConfiguration.class,
  pe.edu.upeu.garritas.common.MessagingConfiguration.class
})
@org.springframework.cloud.openfeign.EnableFeignClients
@SpringBootApplication
public class GarritasVentaMsApplication {
```

Anotaciones presentes en el archivo: @EnableFeignClients, @Import, @SpringBootApplication. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public static void main(String[] args)` · Arranca el contexto de la aplicación; no es el bucle de negocio.

CatalogoConsulta · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/client/CatalogoConsulta.java`.

Envuelve ProductoClient en un Circuit Breaker identificado como catalogo. Traduce producto inexistente a 422 y problemas de disponibilidad a 503. No devuelve un precio inventado cuando falla Catálogo.

```
package pe.edu.upeu.garritas.venta.client;

import feign.FeignException;
import lombok.RequiredArgsConstructor;
import org.springframework.cloud.client.circuitbreaker.CircuitBreakerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.stereotype.Service;
import org.springframework.web.server.ResponseStatusException;

@Service
@RequiredArgsConstructor
public class CatalogoConsulta {
```

Dependencias inyectadas por constructor: ProductoClient client, CircuitBreakerFactory<?, ?> breakers. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public ProductoClient.Producto obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

FeignConfiguration · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/client/FeignConfiguration.java`.

Crea un RequestInterceptor que propaga X-Trace-ID desde MDC a llamadas Feign. La consulta de producto es pública y este interceptor no añade un JWT.

```
package pe.edu.upeu.garritas.venta.client;

import feign.RequestInterceptor;
import org.slf4j.MDC;
import org.springframework.context.annotation.*;

@Configuration(proxyBeanMethods = false)
public class FeignConfiguration {
```

Anotaciones presentes en el archivo: @Bean, @Configuration. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

ProductoClient · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/client/ProductoClient.java`.

Interfaz declarativa OpenFeign con nombre Eureka garritas-catalogo-ms y GET /api/v1/productos/{id}. Su record Producto contiene los datos mínimos de consulta para Venta.

```
package pe.edu.upeu.garritas.venta.client;

import java.math.BigDecimal;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.*;

@FeignClient(name = "garritas-catalogo-ms")
public interface ProductoClient {
```

Anotaciones presentes en el archivo: @FeignClient, @GetMapping, @PathVariable. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `Producto obtener(@PathVariable Long id)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

VentaController · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/controller/VentaController.java`.

Adaptador REST de Venta. Sus mappings declaran rutas, verbo HTTP y respuestas; @Valid activa la validación de entrada. Delega el trabajo en el servicio inyectado. Las restricciones de rol se encuentran además en SecurityConfiguration.

```
package pe.edu.upeu.garritas.venta.controller;

import jakarta.validation.Valid;
import java.util.*;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;
import pe.edu.upeu.garritas.venta.dto.*;
import pe.edu.upeu.garritas.venta.service.VentaService;

@RestController
@RequestMapping("/api/v1/ventas")
@RequiredArgsConstructor
public class VentaController {
```

Dependencias inyectadas por constructor: VentaService service. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @GetMapping, @PathVariable, @PostMapping, @RequestBody, @RequestHeader, @RequestMapping, @RequiredArgsConstructor, @ResponseStatus, @RestController, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<VentaResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public VentaResponse obtener(@PathVariable Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public VentaResponse crear( @Valid @RequestBody VentaRequest request, @RequestHeader("Idempotency-Key") UUID key)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

DetalleVentaRequest · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/dto/DetalleVentaRequest.java`.

DTO de entrada para DetalleVenta. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.venta.dto;

import jakarta.validation.constraints.*;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class DetalleVentaRequest {
```

Anotaciones presentes en el archivo: @Getter, @Max, @Min, @NotNull, @Positive, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Integer cantidad. Sus constraints y relaciones están en el código completo.

DetalleVentaResponse · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/dto/DetalleVentaResponse.java`.

DTO de salida para DetalleVenta. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.venta.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import java.math.BigDecimal;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class DetalleVentaResponse {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, Long idProducto, Integer cantidad, BigDecimal precioUnitario, BigDecimal subtotal. Sus constraints y relaciones están en el código completo.

VentaRequest · garritas-venta-ms

Ubicación: `services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/dto/VentaRequest.java`.

DTO de entrada para Venta. Define exactamente los campos que acepta la API; las anotaciones Bean Validation limitan valores antes de ejecutar la lógica del servicio.

```
package pe.edu.upeu.garritas.venta.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.*;
import java.util.List;
import lombok.Getter;
import lombok.Setter;

@Getter
@Setter
public class VentaRequest {
```

Anotaciones presentes en el archivo: @Getter, @NotEmpty, @NotNull, @Pattern, @Positive, @Setter, @Size, @Valid. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: String metodoPago, List<DetalleVentaRequest> detalles. Sus constraints y relaciones están en el código completo.

VentaResponse · garritas-venta-ms

Ubicación: `services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/dto/VentaResponse.java`.

DTO de salida para Venta. Expone los campos declarados al cliente sin serializar directamente todo el grafo JPA.

```
package pe.edu.upeu.garritas.venta.dto;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
import pe.edu.upeu.garritas.venta.entity.EstadoVenta;

@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class VentaResponse {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Getter, @NoArgsConstructor, @Setter. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, Long sedeId, String idCliente, Instant fecha, EstadoVenta estado, BigDecimal total, List<DetalleVentaResponse> detalles. Sus constraints y relaciones están en el código completo.

DetalleVenta · garritas-venta-ms

Ubicación: `services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/entity/DetalleVenta.java`.

Entidad JPA DetalleVenta: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.venta.entity;

import jakarta.persistence.*;
import lombok.*;

import java.math.BigDecimal;

@Entity
@Table(name = "detalle_ventas")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class DetalleVenta {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Column, @Entity, @GeneratedValue, @Getter, @Id, @JoinColumn, @ManyToOne, @NoArgsConstructor, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, Venta venta, Long idProducto, Integer cantidad, BigDecimal precioUnitario, BigDecimal subtotal. Sus constraints y relaciones están en el código completo.

EstadoVenta · garritas-venta-ms

Ubicación: `services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/entity/EstadoVenta.java`.

Enumeración de valores admitidos por el dominio. Sus constantes impiden usar cadenas arbitrarias en el estado/método de la entidad; las transiciones válidas se controlan en servicios o consumidores.

```
package pe.edu.upeu.garritas.venta.entity;

public enum EstadoVenta {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Los componentes del record o las constantes del enum forman su contrato; consulta el listado completo y la explicación de records/enums.

Venta · garritas-venta-ms

Ubicación: `services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/entity/Venta.java`.

Entidad JPA Venta: representa una tabla del PostgreSQL propio del servicio. Sus campos, relaciones y restricciones se declaran mediante anotaciones; Flyway crea/evoluciona la estructura real. No comparte tablas con otros microservicios.

```
package pe.edu.upeu.garritas.venta.entity;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import lombok.*;

@Entity
@Table(name = "ventas")
@Getter
@Setter
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Venta {
```

Anotaciones presentes en el archivo: @AllArgsConstructor, @Builder, @Column, @Entity, @Enumerated, @GeneratedValue, @Getter, @Id, @NoArgsConstructor, @OneToMany, @Setter, @Table. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

Campos declarados: Long id, String idCliente, Instant fecha, EstadoVenta estado, BigDecimal total, Long sedeId, String metodoPago, java.util.UUID idempotencyKey, List<DetalleVenta> detalles. Sus constraints y relaciones están en el código completo.

GlobalExceptionHandler · garritas-venta-ms

Ubicación: `services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/exception/GlobalExceptionHandler.java`.

Advice REST del módulo: convierte las excepciones de recurso inexistente, validación u otros casos declarados en respuestas HTTP. Se debe leer junto a DomainErrors y a Spring Security; no controla por sí solo todos los fallos del sistema.

```
package pe.edu.upeu.garritas.venta.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;
```

```
@RestControllerAdvice
public class GlobalExceptionHandler {
```

Anotaciones presentes en el archivo: @ExceptionHandler, @RestControllerAdvice. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public ResponseEntity<Map<String, Object>> handleNotFound(ResourceNotFoundException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

• `public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException ex)` · Operación específica de GlobalExceptionHandler. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

ResourceNotFoundException · garritas-venta-ms

Ubicación: `services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/exception/ResourceNotFoundException.java`.

Excepción de dominio que indica que una búsqueda no encontró el recurso. El manejador de excepciones correspondiente la traduce a una respuesta, evitando devolver una entidad nula.

```
package pe.edu.upeu.garritas.venta.exception;

public class ResourceNotFoundException extends RuntimeException {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

CorrelationIdFilter · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/filter/CorrelationIdFilter.java`.

Lee o genera X-Trace-ID, lo asocia al MDC del hilo y devuelve la correlación. Permite buscar en los logs una misma operación HTTP; los eventos conservan traceId. No constituye trazado distribuido automático con OpenTelemetry.

```
package pe.edu.upeu.garritas.venta.filter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.UUID;
import org.slf4j.MDC;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
```

Anotaciones presentes en el archivo: @Component, @Override. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `protected void doFilterInternal( HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)` · Operación específica de CorrelationIdFilter. La firma declara entrada/salida; revisa el cuerpo en el Manual 2 para sus ramas y efectos concretos.

DetalleVentaMapper · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/mapper/DetalleVentaMapper.java`.

Convierte DetalleVenta entre DTO y entidad; separa el contrato HTTP de JPA y establece los valores iniciales que declara su implementación. No debe sustituir las validaciones del servicio.

```
package pe.edu.upeu.garritas.venta.mapper;

import org.springframework.stereotype.Component;
import pe.edu.upeu.garritas.venta.dto.DetalleVentaResponse;
import pe.edu.upeu.garritas.venta.entity.DetalleVenta;

@Component
public class DetalleVentaMapper {
```

Anotaciones presentes en el archivo: @Component. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public DetalleVentaResponse toResponse(DetalleVenta detalle)` · Proyecta campos de entidad a DTO de salida.

VentaMapper · garritas-venta-ms

Ubicación: `services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/mapper/VentaMapper.java`.

Convierte Venta entre DTO y entidad; separa el contrato HTTP de JPA y establece los valores iniciales que declara su implementación. No debe sustituir las validaciones del servicio.

```
package pe.edu.upeu.garritas.venta.mapper;

import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;
import pe.edu.upeu.garritas.venta.dto.VentaResponse;
import pe.edu.upeu.garritas.venta.entity.Venta;

@Component
@RequiredArgsConstructor
public class VentaMapper {
```

Dependencias inyectadas por constructor: DetalleVentaMapper detalleVentaMapper. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @Component, @RequiredArgsConstructor. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public VentaResponse toResponse(Venta venta)` · Proyecta campos de entidad a DTO de salida.

VentaEvents · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/messaging/VentaEvents.java`.

Consume resultados de inventario y pago, descarta tipos ajenos, deduplica mediante Inbox y actualiza el estado de la venta según las transiciones permitidas. La compensación solo termina al recibir inventario.liberado.

```
package pe.edu.upeu.garritas.venta.messaging;

import lombok.RequiredArgsConstructor;
import org.slf4j.*;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;
import pe.edu.upeu.garritas.common.*;
import tools.jackson.databind.ObjectMapper;

@Component
@RequiredArgsConstructor
public class VentaEvents {
```

Dependencias inyectadas por constructor: JdbcTemplate jdbc, Inbox inbox, ObjectMapper json. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @Component, @KafkaListener, @RequiredArgsConstructor, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public void consume(String payload)` · Punto de entrada Kafka: deserializa contrato, filtra tipo, deduplica y ejecuta el efecto local.

VentaRepository · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/repository/VentaRepository.java`.

Puerto de persistencia Spring Data de Venta. La interfaz y sus métodos declaran búsquedas, bloqueos o grafos de lectura; Spring genera la implementación al iniciar el contexto.

```
package pe.edu.upeu.garritas.venta.repository;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import pe.edu.upeu.garritas.venta.entity.Venta;

public interface VentaRepository extends JpaRepository<Venta, Long> {
```

Anotaciones presentes en el archivo: ninguna. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `java.util.List<Venta> findByIdCliente(String idCliente)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

• `java.util.Optional<Venta> findByIdClienteAndIdempotencyKey(String idCliente, java.util.UUID key)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

• `List<Venta> findByIdCliente(Long idCliente)` · Contrato de interfaz; la implementación proviene del framework o de la clase declarada, según el tipo de interfaz.

VentaService · garritas-venta-ms

Ubicación:
`services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/service/VentaService.java`.

Valida sede activa y precios/stock remotos. Serializa una misma Idempotency-Key por sub con pg_advisory_xact_lock; rechaza cambios de payload para una clave ya usada. Guarda la venta y venta.creada en la misma transacción; la respuesta inicial no implica pago completado.

```
package pe.edu.upeu.garritas.venta.service;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.*;
import lombok.RequiredArgsConstructor;
import org.slf4j.MDC;
import org.springframework.http.HttpStatus;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.server.ResponseStatusException;
import pe.edu.upeu.garritas.common.*;
import pe.edu.upeu.garritas.venta.client.CatalogoConsulta;
import pe.edu.upeu.garritas.venta.dto.*;
import pe.edu.upeu.garritas.venta.entity.*;
import pe.edu.upeu.garritas.venta.mapper.VentaMapper;
import pe.edu.upeu.garritas.venta.repository.VentaRepository;

@Service
```

```
@RequiredArgsConstructor
public class VentaService {
```

Dependencias inyectadas por constructor: VentaRepository repository, VentaMapper mapper, CatalogoConsulta catalogo, Outbox outbox, JdbcTemplate jdbc, ServiceClient services. El bean que las crea es Spring; no se construye otro repositorio con new en cada petición.

Anotaciones presentes en el archivo: @RequiredArgsConstructor, @Service, @Transactional. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

• `public List<VentaResponse> listar()` · Obtiene una colección; comprueba el filtro por propietario/rol y el orden del código.

• `public VentaResponse obtener(Long id)` · Busca un recurso por id y lo convierte al contrato de salida; los servicios propietarios también autorizan su acceso.

• `public VentaResponse crear(VentaRequest request, UUID key)` · Valida y construye el recurso, persiste localmente y devuelve su DTO; los efectos remotos dependen de la implementación.

• `private Venta owned(Long id)` · Busca el recurso y compara propietario con sub, permitiendo auditoría ADMIN.

GarritasVentaMsApplicationTests · garritas-venta-ms

Ubicación: `services/garritas-venta-ms/src/test/java/pe/edu/upeu/garritas/venta/GarritasVentaMsApplicationTests.java`.

Prueba de carga del contexto Spring de este módulo. Verifica que sus beans y configuración pueden iniciarse contra el entorno de pruebas; no demuestra por sí sola los flujos distribuidos.

```
package pe.edu.upeu.garritas.venta;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class GarritasVentaMsApplicationTests {
```

Anotaciones presentes en el archivo: @SpringBootTest, @Test. Consulta sus efectos en el capítulo de anotaciones; no todas corresponden al encabezado de clase.

La responsabilidad se configura por anotaciones o por métodos con firma repartida en líneas; consulta el archivo completo del Manual 2. No se infiere una operación CRUD que no esté declarada.

15. Fichas del frontend TypeScript

app.config.ts

Ubicación: `clients/garritas-frontend/src/app/app.config.ts`.

Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import {
  ApplicationConfig,
  inject,
  provideAppInitializer,
  provideBrowserGlobalErrorListeners,
} from "@angular/core";
import { provideHttpClient, withInterceptors } from "@angular/common/http";
import { Auth } from "./core/auth";
import { authInterceptor } from "./core/interceptor";
export const appConfig: ApplicationConfig = {
```

app.ts

Ubicación: `clients/garritas-frontend/src/app/app.ts`.

App: Componente raíz standalone: coordina navegación, catálogo, carrito, sesión, pedidos/polling y vistas de cada dominio. Oculta administración para clientes y verifica la vista guardada al restaurar la sesión; backend mantiene la autorización efectiva.

```
import { DomainApi, Sede, stateLabel } from "./core/domain-api";
import { SedesComponent } from "./features/sedes/sedes.component";
import { EspecialistasComponent } from "./features/bienestar/especialistas.component";
import { CuentaComponent } from "./features/cuenta/cuenta.component";
import { CitasComponent } from "./features/citas/citas.component";
import { EnviosComponent } from "./features/envios/envios.component";
import { PagosComponent } from "./features/pagos/pagos.component";
import { ProductosComponent } from "./features/admin/productos.component";
import { ClientesComponent } from "./features/admin/clientes.component";
import { Component, DestroyRef, inject, signal, effect } from "@angular/core";
import { CurrencyPipe, DatePipe } from "@angular/common";
import { FormsModule } from "@angular/forms";
import { takeUntilDestroyed } from "@angular/core/rxjs-interop";
import { timer, EMPTY, exhaustMap, catchError } from "rxjs";
import { HttpErrorResponse } from "@angular/common/http";
import { Auth } from "./core/auth";
import { Api, Categoria, Producto, Venta, Pago, pending } from "./core/api";
import { CartStore } from "./features/cart/cart-store";
import { CartPanel } from "./features/cart/cart-panel";
import { ProductCatalog } from "./features/catalogo/product-catalog";

@Component({
  selector: "app-root",
  imports: [
    FormsModule,
    CurrencyPipe,
    DatePipe,
    CartPanel,
    ProductCatalog,
    SedesComponent,
    EspecialistasComponent,
    CuentaComponent,
    CitasComponent,
    EnviosComponent,
    PagosComponent,
    ProductosComponent,
    ClientesComponent,
  ],
  templateUrl: "./app.html",
})
export class App {
```

Inyección: auth ← Auth, cart ← CartStore, api ← Api, domain ← DomainApi, destroy ← DestroyRef.

Métodos escritos: constructor, refreshSedes, openCitas, shipping, refreshCatalogo, refreshVentas, add, checkout, consultarPago, edit, save, remove, fail. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen

completas en el Manual 2.

api.spec.ts

Ubicación: `clients/garritas-frontend/src/app/core/api.spec.ts`.

Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { describe, it, expect } from "vitest";
import { pending } from "./api";
describe("Estados de consistencia eventual", () => {
  it("sigue consultando hasta confirmar pago o compensación", () => {
    for (const state of ["PENDIENTE_STOCK", "PENDIENTE_PAGO", "COMPENSANDO"])
      expect(pending(state)).toBe(true);
    for (const state of ["PAGADA", "CANCELADA"])
      expect(pending(state)).toBe(false);
  });
});
```

Fuente de pruebas, no componente de pantalla. Los casos it/test describen el comportamiento que debe mantenerse y los dobles HTTP aíslan las respuestas para controlar la secuencia.

api.ts

Ubicación: `clients/garritas-frontend/src/app/core/api.ts`.

Api: Servicio HttpClient tipado para catálogo, categorías, ventas y pago por venta. crearVenta añade Idempotency-Key; el interceptor incorpora el JWT solo para la API propia.

```
import { Injectable, inject } from "@angular/core";
import { HttpClient } from "@angular/common/http";
import { Config } from "./config";
export interface Categoria {
```

Inyección: http ← HttpClient, config ← Config.

Métodos escritos: productos, categorias, borrarCategoria, ventas, pago. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

Interfaces de contrato: Categoria, Producto, Venta, Pago. Son tipos de compilación y desaparecen en el JavaScript emitido; el backend mantiene la validación efectiva.

auth.ts

Ubicación: `clients/garritas-frontend/src/app/core/auth.ts`.

Auth: Servicio singleton Angular que inicializa keycloak-js con check-sso y PKCE S256, mantiene signals de sesión/rol, renueva tokens y ejecuta login/logout. Los tokens permanecen en memoria.

```
import { Injectable, inject, signal } from "@angular/core";
import Keycloak from "keycloak-js";
import { Config } from "./config";
@Injectable({ providedIn: "root" })
export class Auth {
```

Inyección: config ← Config.

Métodos escritos: init, sync, login, logout, token. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

config.ts

Ubicación: `clients/garritas-frontend/src/app/core/config.ts`.

Config: Carga config.dev.json en desarrollo o config.json en producción antes de inicializar la autenticación. Permite cambiar URLs de API y Keycloak sin insertar esas direcciones en cada componente.

```
import { Injectable, isDevMode } from "@angular/core";
export interface RuntimeConfig {
```

Métodos escritos: load. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

Interfaces de contrato: RuntimeConfig. Son tipos de compilación y desaparecen en el JavaScript emitido; el backend mantiene la validación efectiva.

domain-api.ts

Ubicación: `clients/garritas-frontend/src/app/core/domain-api.ts`.

DomainApi: Servicio HTTP genérico y tipos para sedes, profesionales, especialidades, cuenta, mascotas, citas, envíos y administración; centraliza el prefijo de API configurado.

Feedback: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Injectable, inject, signal } from "@angular/core";
import { HttpClient, HttpErrorResponse } from "@angular/common/http";
import { Observable } from "rxjs";
import { Config } from "./config";
export interface Sede {
```

Inyección: http ← HttpClient, config ← Config.

Métodos escritos: delete, report. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

Interfaces de contrato: Sede, Especialidad, Profesional, Perfil, Mascota, Cita, Envio, PagoDetalle. Son tipos de compilación y desaparecen en el JavaScript emitido; el backend mantiene la validación efectiva.

interceptor.ts

Ubicación: `clients/garritas-frontend/src/app/core/interceptor.ts`.

Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { HttpInterceptorFn } from "@angular/common/http";
import { inject } from "@angular/core";
import { from, switchMap } from "rxjs";
import { Auth } from "./auth";
import { Config } from "./config";
export const authInterceptor: HttpInterceptorFn = (req, next) => {
```

Inyección: config ← Config, auth ← Auth.

clientes.component.ts

Ubicación: `clients/garritas-frontend/src/app/features/admin/clientes.component.ts`.

ClientesComponent: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Component, inject, signal } from "@angular/core";
import { FormsModule } from "@angular/forms";
import { DomainApi, Feedback, Perfil } from "../../core/domain-api";
@Component({
  selector: "app-clientes",
  imports: [FormsModule],
  templateUrl: "./clientes.component.html",
})
```

```
export class ClientesComponent extends Feedback {
```

Inyección: api ← DomainApi.

Métodos escritos: constructor, load, edit, save, remove. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

productos.component.ts

Ubicación: `clients/garritas-frontend/src/app/features/admin/productos.component.ts`.

ProductosComponent: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Component, inject, signal } from "@angular/core";
import { FormsModule } from "@angular/forms";
import { CurrencyPipe } from "@angular/common";
import { forkJoin } from "rxjs";
import { DomainApi, Feedback } from "../../core/domain-api";
import { Producto, Categoria } from "../../core/api";
@Component({
  selector: "app-admin-productos",
  imports: [FormsModule, CurrencyPipe],
  templateUrl: "./productos.component.html",
})
export class ProductosComponent extends Feedback {
```

Inyección: api ← DomainApi.

Métodos escritos: constructor, load, edit, reset, save, deactivate. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

especialistas.component.ts

Ubicación: `clients/garritas-frontend/src/app/features/bienestar/especialistas.component.ts`.

EspecialistasComponent: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Component, inject, signal, output } from "@angular/core";
import { FormsModule } from "@angular/forms";
import { forkJoin } from "rxjs";
import { Auth } from "../../core/auth";
import {
  DomainApi,
  Feedback,
  Sede,
  Especialidad,
  Profesional,
} from "../../core/domain-api";
@Component({
  selector: "app-especialistas",
  imports: [FormsModule],
  templateUrl: "./especialistas.component.html",
})
export class EspecialistasComponent extends Feedback {
```

Inyección: auth ← Auth, api ← DomainApi.

Métodos escritos: constructor, load, sede, edit, toggle, save, deactivate, editEsp, saveEsp, deleteEsp. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

cart-panel.ts

Ubicación: `clients/garritas-frontend/src/app/features/cart/cart-panel.ts`.

CartPanel: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Component, inject, input, output, signal } from "@angular/core";
import { CurrencyPipe } from "@angular/common";
import { FormsModule } from "@angular/forms";
import { CartStore } from "./cart-store";

@Component({
  selector: "app-cart-panel",
  imports: [CurrencyPipe, FormsModule],
  templateUrl: "./cart-panel.html",
})
export class CartPanel {
```

Inyección: cart ← CartStore.

Métodos escritos: change. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

cart-store.spec.ts

Ubicación: `clients/garritas-frontend/src/app/features/cart/cart-store.spec.ts`.

Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { beforeEach, describe, expect, it } from "vitest";
import { Producto } from "../../core/api";
import { CART_STORAGE_KEY, CartStore } from "./cart-store";

const food: Producto = {
  id: 1,
  nombre: "Alimento para perro",
  descripcion: "",
  precio: 10.1,
  stock: 3,
  activo: true,
  categoria: { id: 1, nombre: "Alimentos", descripcion: "" },
};
const toy: Producto = { ...food, id: 2, nombre: "Pelota", precio: 20.2 };
const catalog = [food, toy];
const sedes = [
  {
    id: 1,
    nombre: "Miraflores",
    direccion: "Dirección de ejemplo",
```

Fuente de pruebas, no componente de pantalla. Los casos it/test describen el comportamiento que debe mantenerse y los dobles HTTP aíslan las respuestas para controlar la secuencia.

cart-store.ts

Ubicación: `clients/garritas-frontend/src/app/features/cart/cart-store.ts`.

CartStore: Estado del carrito con signals y persistencia local de ids, cantidades, sede, método de pago y clave de reintento. Revalida stock/sede activa y prepara el payload sin precios confiables del cliente. No almacena tokens.

```
import { Injectable, computed, signal } from "@angular/core";
import { Sede } from "../../core/domain-api";
import { Producto } from "../../core/api";

export interface CartEntry {
```

Métodos escritos: constructor, reconcile, setSedes, selectSede, remove, clear, setPayment, payload, set, persist, restore. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

Interfaces de contrato: CartEntry, CartLine. Son tipos de compilación y desaparecen en el JavaScript emitido; el backend mantiene la validación efectiva.

product-catalog.ts

Ubicación: `clients/garritas-frontend/src/app/features/catalogo/product-catalog.ts`.

ProductCatalog: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Component, computed, input, output, signal } from "@angular/core";
import { CurrencyPipe } from "@angular/common";
import { FormsModule } from "@angular/forms";
import { Categoria, Producto } from "../../core/api";
@Component({
  selector: "app-product-catalog",
  imports: [CurrencyPipe, FormsModule],
  templateUrl: "./product-catalog.html",
})
export class ProductCatalog {
```

Métodos escritos: icon. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

citas.component.ts

Ubicación: `clients/garritas-frontend/src/app/features/citas/citas.component.ts`.

CitasComponent: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Component, inject, signal, output } from "@angular/core";
import { FormsModule } from "@angular/forms";
import { DatePipe } from "@angular/common";
import { Auth } from "../../core/auth";
import {
  DomainApi,
  Feedback,
  Sede,
  Profesional,
  Mascota,
  Cita,
  stateLabel,
} from "../../core/domain-api";
import { CartStore } from "../cart/cart-store";
@Component({
  selector: "app-citas",
  imports: [FormsModule, DatePipe],
  templateUrl: "./citas.component.html",
})
export class CitasComponent extends Feedback {
```

Inyección: auth ← Auth, cart ← CartStore, api ← DomainApi.

Métodos escritos: constructor, load, available, save, action. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

cuenta.component.spec.ts

Ubicación: `clients/garritas-frontend/src/app/features/cuenta/cuenta.component.spec.ts`.

Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { TestBed } from '@angular/core/testing';
import { provideHttpClient } from '@angular/common/http';
import { provideHttpClientTesting, HttpTestingController } from '@angular/common/http/testing';
import { CuentaComponent } from './cuenta.component';
import { Config } from '../../core/config';
import { describe, it, expect, afterEach } from 'vitest';

describe('Cuenta: carga del perfil antes de editar', () => {
  afterEach(() => TestBed.inject(HttpTestingController).verify());
  function setup() {
    TestBed.configureTestingModule({
```

```
      imports: [CuentaComponent],
      providers: [provideHttpClient(), provideHttpClientTesting(),
        { provide: Config, useValue: { value: { apiUrl: '/api/v1' } } }],
    });
    const fixture = TestBed.createComponent(CuentaComponent);
    fixture.detectChanges();
    return { fixture, http: TestBed.inject(HttpTestingController) };
  }
  it('no permite editar mientras una respuesta tardía pueda reemplazar los campos', async () => {
```

Fuente de pruebas, no componente de pantalla. Los casos it/test describen el comportamiento que debe mantenerse y los dobles HTTP aíslan las respuestas para controlar la secuencia.

cuenta.component.ts

Ubicación: `clients/garritas-frontend/src/app/features/cuenta/cuenta.component.ts`.

CuentaComponent: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Component, inject, signal } from "@angular/core";
import { FormsModule } from "@angular/forms";
import { DomainApi, Feedback, Perfil, Mascota } from "../../core/domain-api";
@Component({
  selector: "app-cuenta",
  imports: [FormsModule],
  templateUrl: "./cuenta.component.html",
})
export class CuentaComponent extends Feedback {
```

Inyección: api ← DomainApi.

Métodos escritos: constructor, load, loadPets, save, savePet, edit, remove. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

envios.component.ts

Ubicación: `clients/garritas-frontend/src/app/features/envios/envios.component.ts`.

EnviosComponent: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Component, inject, signal, input, OnInit } from "@angular/core";
import { FormsModule } from "@angular/forms";
import { DatePipe } from "@angular/common";
import { forkJoin } from "rxjs";
import { Auth } from "../../core/auth";
import { Venta } from "../../core/api";
import { DomainApi, Feedback, Envio, stateLabel } from "../../core/domain-api";
@Component({
  selector: "app-envios",
  imports: [FormsModule, DatePipe],
  templateUrl: "./envios.component.html",
})
export class EnviosComponent extends Feedback implements OnInit {
```

Inyección: auth ← Auth, api ← DomainApi.

Métodos escritos: ngOnInit, load, available, save, action. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

pagos.component.ts

Ubicación: `clients/garritas-frontend/src/app/features/pagos/pagos.component.ts`.

PagosComponent: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Component, inject, signal } from "@angular/core";
import { CurrencyPipe, DatePipe } from "@angular/common";
import {
  DomainApi,
  Feedback,
  PagoDetalle,
  stateLabel,
} from "../../core/domain-api";
@Component({
  selector: "app-pagos",
  imports: [CurrencyPipe, DatePipe],
  template: `<div class="page-heading">
      <div>
        <p class="eyebrow">TUS OPERACIONES</p>
        <h1>Pagos</h1>
        <p>
          Resultados de pago asociados a tus pedidos. El procesamiento es de
          demostración.
        </p>
      </div>
      <button (click)="load()">Actualizar pagos</button>
    </div>
    @if (error()) {
      <p class="error" role="alert">{{ error() }}</p>
    }
    <div class="table-wrap">
      <table>
        <thead>
          <tr>
            <th>Pago</th>
            <th>Pedido</th>
            <th>Fecha</th>
            <th>Método</th>
            <th>Monto</th>
            <th>Estado</th>
          </tr>
        </thead>
        <tbody>
          @for (p of pagos(); track p.id) {
            <tr>
              <td>#{{ p.id }}</td>
              <td>#{{ p.idVenta }}</td>
              <td>{{ p.fechaPago | date: "short" }}</td>
              <td>{{ p.metodo }}</td>
              <td>{{ p.monto | currency: "PEN" : "S/ " }}</td>
              <td>
                <span class="badge">{{ label(p.estado) }}</span>
              </td>
            </tr>
          } @empty {
            <tr>
              <td colspan="6">Todavía no hay pagos procesados.</td>

            </tr>
          }
        </tbody>
      </table>
    </div>`,
})
export class PagosComponent extends Feedback {
```

Inyección: api ← DomainApi.

Métodos escritos: constructor, load. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

sedes.component.ts

Ubicación: `clients/garritas-frontend/src/app/features/sedes/sedes.component.ts`.

SedesComponent: Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { Component, inject, signal, output } from "@angular/core";
import { FormsModule } from "@angular/forms";
import { Auth } from "../../core/auth";
import { DomainApi, Feedback, Sede } from "../../core/domain-api";
import { CartStore } from "../cart/cart-store";
@Component({
```

```
  selector: "app-sedes",
  imports: [FormsModule],
  templateUrl: "./sedes.component.html",
})
export class SedesComponent extends Feedback {
```

Inyección: auth ← Auth, cart ← CartStore, api ← DomainApi.

Métodos escritos: constructor, load, edit, reset, save, deactivate, choose. load/refrescar consulta la API; save/envío usa Feedback; edit prepara modelo del formulario; remove solicita confirmación y baja; las operaciones concretas se leen completas en el Manual 2.

main.ts

Ubicación: `clients/garritas-frontend/src/main.ts`.

Fuente Angular: lógica, tipos y estado del componente/servicio. Se compila con Angular CLI; los datos de negocio proceden de la API.

```
import { bootstrapApplication } from '@angular/platform-browser';
import { App } from './app/app';
import { appConfig } from './app/app.config';
bootstrapApplication(App,appConfig).catch(console.error);
```

16. Configuraciones y automatizaciones: cómo encajan

POM Maven

Dependencias, versión, plugins y módulos; resueltos antes de ejecutar.

- `infra/garritas-common/pom.xml`
- `infra/garritas-config/pom.xml`
- `infra/garritas-eureka/pom.xml`
- `infra/garritas-gateway/pom.xml`
- `services/garritas-bienestar-ms/pom.xml`
- `services/garritas-catalogo-ms/pom.xml`
- `services/garritas-citas-ms/pom.xml`
- `services/garritas-envio-ms/pom.xml`
- `services/garritas-institucion-ms/pom.xml`
- `services/garritas-pago-ms/pom.xml`
- `services/garritas-usuario-ms/pom.xml`
- `services/garritas-venta-ms/pom.xml`
- `uso-rapido/pom.xml`

Application y Config Repo

application.yml contiene identidad/importación; config-repo contiene valores por perfil.

- `infra/garritas-config/config-repo/garritas-bienestar-ms-dev.yml`
- `infra/garritas-config/config-repo/garritas-bienestar-ms-prod.yml`
- `infra/garritas-config/config-repo/garritas-catalogo-ms-dev.yml`
- `infra/garritas-config/config-repo/garritas-catalogo-ms-prod.yml`
- `infra/garritas-config/config-repo/garritas-citas-ms-dev.yml`
- `infra/garritas-config/config-repo/garritas-citas-ms-prod.yml`
- `infra/garritas-config/config-repo/garritas-envio-ms-dev.yml`
- `infra/garritas-config/config-repo/garritas-envio-ms-prod.yml`
- `infra/garritas-config/config-repo/garritas-eureka-dev.yml`
- `infra/garritas-config/config-repo/garritas-eureka-prod.yml`
- `infra/garritas-config/config-repo/garritas-gateway-dev.yml`
- `infra/garritas-config/config-repo/garritas-gateway-prod.yml`
- `infra/garritas-config/config-repo/garritas-institucion-ms-dev.yml`

- infra/garritas-config/config-repo/garritas-institucion-ms-prod.yml
- infra/garritas-config/config-repo/garritas-pago-ms-dev.yml
- infra/garritas-config/config-repo/garritas-pago-ms-prod.yml
- infra/garritas-config/config-repo/garritas-usuario-ms-dev.yml
- infra/garritas-config/config-repo/garritas-usuario-ms-prod.yml
- infra/garritas-config/config-repo/garritas-venta-ms-dev.yml
- infra/garritas-config/config-repo/garritas-venta-ms-prod.yml
- infra/garritas-config/src/main/resources/application.yml
- infra/garritas-eureka/src/main/resources/application.yml
- infra/garritas-gateway/src/main/resources/application.yml
- services/garritas-bienestar-ms/src/main/resources/application.yml
- services/garritas-catalogo-ms/src/main/resources/application.yml
- services/garritas-citas-ms/src/main/resources/application.yml
- services/garritas-envio-ms/src/main/resources/application.yml
- services/garritas-institucion-ms/src/main/resources/application.yml
- services/garritas-pago-ms/src/main/resources/application.yml
- services/garritas-usuario-ms/src/main/resources/application.yml
- services/garritas-venta-ms/src/main/resources/application.yml

Docker/Compose/Nginx

Empaquetado, redes, puertos, volúmenes y proxy; DEV e integrado son modalidades diferentes.

- clients/garritas-frontend/Dockerfile
- clients/garritas-frontend/Dockerfile.prebuilt
- clients/garritas-frontend/Dockerfile.prebuilt.dockerignore
- clients/garritas-frontend/nginx.conf
- infra/compose.yml
- infra/garritas-config/Dockerfile
- infra/garritas-eureka/Dockerfile
- infra/garritas-gateway/Dockerfile
- kafka/compose-dev.yml
- keycloak/compose-dev.yml
- obs/compose-dev.yml
- obs/compose.yml
- services/garritas-bienestar-ms/Dockerfile

- services/garritas-bienestar-ms/compose-dev.yml
- services/garritas-bienestar-ms/compose.yml
- services/garritas-catalogo-ms/Dockerfile
- services/garritas-catalogo-ms/compose-dev.yml
- services/garritas-catalogo-ms/compose.yml
- services/garritas-citas-ms/Dockerfile
- services/garritas-citas-ms/compose-dev.yml
- services/garritas-citas-ms/compose.yml
- services/garritas-envio-ms/Dockerfile
- services/garritas-envio-ms/compose-dev.yml
- services/garritas-envio-ms/compose.yml
- services/garritas-institucion-ms/Dockerfile
- services/garritas-institucion-ms/compose-dev.yml
- services/garritas-institucion-ms/compose.yml
- services/garritas-pago-ms/Dockerfile
- services/garritas-pago-ms/compose-dev.yml
- services/garritas-pago-ms/compose.yml
- services/garritas-usuario-ms/Dockerfile
- services/garritas-usuario-ms/compose-dev.yml
- services/garritas-usuario-ms/compose.yml
- services/garritas-venta-ms/Dockerfile
- services/garritas-venta-ms/compose-dev.yml
- services/garritas-venta-ms/compose.yml
- uso-rapido/docker/Dockerfile
- uso-rapido/docker/Dockerfile.dockerignore
- uso-rapido/docker/Dockerfile.runtime
- uso-rapido/docker/Dockerfile.runtime.dockerignore
- uso-rapido/docker/compose.observability.yml
- uso-rapido/docker/compose.prebuilt.yml
- uso-rapido/docker/compose.yml
- uso-rapido/garritas-eventos-py/Dockerfile
- uso-rapido/garritas-eventos-py/compose.yml

Observabilidad

Targets, scrapes, labels, pipelines y provisión Grafana.

- `obs/.gitkeep`
- `obs/grafana/provisioning/alerting/garritas-alertas.yml`
- `obs/grafana/provisioning/dashboards/dashboards.yml`
- `obs/grafana/provisioning/dashboards/garritas-catalogo.json`
- `obs/grafana/provisioning/dashboards/garritas-infraestructura.json`
- `obs/grafana/provisioning/datasources/datasources.yml`
- `obs/prometheus/prometheus-dev.yml`
- `obs/prometheus/prometheus.yml`
- `obs/promtail/promtail-config-dev.yml`
- `obs/promtail/promtail-config.yml`

Frontend build/runtime

CLI, TypeScript, npm y URLs runtime; config.dev.json difiere de config.json.

- `clients/garritas-frontend/angular.json`
- `clients/garritas-frontend/package.json`
- `clients/garritas-frontend/public/config.dev.json`
- `clients/garritas-frontend/public/config.json`
- `clients/garritas-frontend/tsconfig.app.json`
- `clients/garritas-frontend/tsconfig.json`
- `clients/garritas-frontend/tsconfig.spec.json`

Herramientas y CI

Arranque/status y pruebas externas; el script no sustituye invariantes del backend.

- `.github/workflows/ci.yml`
- `uso-rapido/scripts/consoles.py`
- `uso-rapido/scripts/dev.sh`
- `uso-rapido/scripts/domain_smoke.py`
- `uso-rapido/scripts/faults.py`
- `uso-rapido/scripts/observability.sh`
- `uso-rapido/scripts/requirements-test.txt`
- `uso-rapido/scripts/smoke.py`
- `uso-rapido/scripts/status.py`

Identidad

Realm, cliente, roles, usuarios y audiencia iniciales de laboratorio.

- `keycloak/garritas-realm.json`

17. Catálogo HTTP por controlador

Los fragmentos siguientes son los mappings escritos. El prefijo de clase se combina con el mapping del método; /api/v1 es común. Los permisos se deciden por SecurityConfiguration y propiedad en servicio, no por la palabra Controller.

BienestarController

`services/garritas-bienestar-ms/src/main/java/pe/edu/upeu/garritas/bienestar/controller/BienestarController.java`

```
@RequestMapping("/api/v1")
@GetMapping("/especialistas")
@GetMapping("/especialistas/{id}")
@PostMapping("/especialistas")
@PutMapping("/especialistas/{id}")
@DeleteMapping("/especialistas/{id}")
@GetMapping("/especialidades")
@PostMapping("/especialidades")
@PutMapping("/especialidades/{id}")
@DeleteMapping("/especialidades/{id}")
```

CategoriaController

`services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/controller/CategoriaController.java`

```
@RequestMapping("/api/v1/categorias")
@GetMapping
@GetMapping("/{id}")
@PostMapping
@PutMapping("/{id}")
@DeleteMapping("/{id}")
```

ProductoController

`services/garritas-catalogo-ms/src/main/java/pe/edu/upeu/garritas/catalogo/controller/ProductoController.java`

```
@RequestMapping("/api/v1/productos")
@GetMapping
@GetMapping("/{id}")
@PostMapping
@PutMapping("/{id}")
@DeleteMapping("/{id}")
```

CitaController

`services/garritas-citas-ms/src/main/java/pe/edu/upeu/garritas/citas/controller/CitaController.java`

```
@RequestMapping("/api/v1/citas")
@GetMapping
@GetMapping("/{id}")
@PostMapping
@PostMapping("/{id}/confirmar")
@PostMapping("/{id}/atender")
@PostMapping("/{id}/cancelar")
```

EnvioController

`services/garritas-envio-ms/src/main/java/pe/edu/upeu/garritas/envio/controller/EnvioController.java`

```
@RequestMapping("/api/v1/envios")
@GetMapping
@GetMapping("/{id}")
@PostMapping
```

```
@PostMapping("/{id}/en-transito")
@PostMapping("/{id}/entregado")
@PostMapping("/{id}/cancelar")
```

SedeController

services/garritas-institucion-ms/src/main/java/pe/edu/upeu/garritas/institucion/controller/SedeController.java

```
@RequestMapping("/api/v1/sedes")
@GetMapping
@GetMapping("/{id}")
@PostMapping
@PutMapping("/{id}")
@DeleteMapping("/{id}")
```

PagoController

services/garritas-pago-ms/src/main/java/pe/edu/upeu/garritas/pago/controller/PagoController.java

```
@RequestMapping("/api/v1/pagos")
@GetMapping
@GetMapping("/{id}")
@GetMapping("/venta/{id}")
```

MascotaController

services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/MascotaController.java

```
@RequestMapping("/api/v1/usuarios/{usuarioId}/mascotas")
@GetMapping
@PostMapping
@PutMapping("/{id}")
@DeleteMapping("/{id}")
```

MiCuentaController

services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/MiCuentaController.java

```
@RequestMapping("/api/v1/usuarios/me")
@GetMapping
@PutMapping
@GetMapping("/mascotas")
@GetMapping("/mascotas/{id}")
@PostMapping("/mascotas")
@PutMapping("/mascotas/{id}")
@DeleteMapping("/mascotas/{id}")
```

UsuarioController

services/garritas-usuario-ms/src/main/java/pe/edu/upeu/garritas/usuario/controller/UsuarioController.java

```
@RequestMapping("/api/v1/usuarios")
@GetMapping
@GetMapping("/{id}")
@PostMapping
@PutMapping("/{id}")
@DeleteMapping("/{id}")
```

VentaController

services/garritas-venta-ms/src/main/java/pe/edu/upeu/garritas/venta/controller/VentaController.java

```
@RequestMapping("/api/v1/ventas")
@GetMapping
@GetMapping("/{id}")
@PostMapping
```

18. Qué no debes confundir al explicar el proyecto

• Ocho microservicios de negocio no significa ocho procesos totales: hay once Java con infraestructura, Angular y contenedores auxiliares.

• Eureka descubre; Gateway enruta; Config suministra propiedades. Ninguno reemplaza a los otros.

• Autenticación, rol, propiedad y CORS son controles distintos y complementarios.

• M:N es una relación local en Bienestar; una referencia HTTP entre dominios no es una FK entre bases.

• Saga no es una transacción ACID global ni una promesa de devolución bancaria.

• Un volumen persiste datos, pero no sustituye backup ni réplica.

• Kafka es asíncrono; un 201 de venta no implica PAGADA.

• Readiness y métricas UP no equivalen a cobertura total funcional.

• GitHub aloja código; localhost señala tu propia máquina, no una web pública.

• DEV validado no certifica Windows ni el Compose integral no ejecutado.

19. Lecturas y ubicación de las fuentes

• Referencia docente Pagatu: https://github.com/262dist/pagatu · se usa como patrón de estructura y progreso, adaptado al dominio mascotas.

• Repositorio Garritas: https://github.com/Nikol235/Garritas- · rama feat/garritas-store.

• Guías del proyecto: README.md, docs/arquitectura.md, docs/ejecucion.md y docs/validacion.md.

• Documentación oficial de Spring Boot/Cloud/Security/Kafka, PostgreSQL, Angular, Keycloak, Kafka, Docker, Prometheus y Grafana: consulta la versión fijada por los manifiestos antes de aplicar instrucciones de otra generación.

• Manual 1: comandos/puertos y evidencia; Manual 2: reconstrucción y código íntegro; este Manual 3: interpretación y conexiones.